

JavaScript – Complete Theory Guide (Zero to Interview Ready)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 17 Aug 2026 – 6 Sep 2026 (Days 1-21)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import <calendar/mern-mastery-study.ics>

MERN Bootcamp — Module 01 | Target: 30 LPA+ Roles in India (2026)

Beginner? Open [00-beginner-glossary.pdf](#) first — every term is written in full (no unexplained short forms like "TDZ" or "REPL").

How to Use This Guide

Who This Is For	What You'll Gain
Absolute beginners with zero programming background	Solid JS fundamentals that survive senior-level interviews
Bootcamp students preparing for MERN stack	Deep theory before React, Node, Express, MongoDB
Working developers targeting 30 LPA+ in India	Interview-ready depth on closures, event loop, async, prototypes

Goal: By the end of this document, you should be able to explain *why* JavaScript behaves the way it does — not just *what* the syntax looks like. Indian product companies (Flipkart, Razorpay, Swiggy, PhonePe, Atlassian India, etc.) and service-based MNCs (TCS Digital, Infosys, Wipro) test JavaScript fundamentals heavily in DSA rounds, machine coding, and system design discussions.

How to study (6-month bootcamp pacing):

1. **Week 1–2:** Sections 1–7 (History → Closures). Code every example in VS Code + browser console.
2. **Week 3:** Sections 8–12 (this → Objects/Arrays). Build a mini todo app using only vanilla JS.
3. **Week 4:** Sections 13–17 (Errors → async/await). Solve 20 LeetCode Easy/Medium in JS.
4. **Week 5:** Sections 18–19 (Modules → Utility patterns). Implement debounce/throttle/curry from scratch.
5. **Week 6:** Section 20 (Interview bank) + timed mock interviews.

Rules while studying:

- Never copy-paste without typing and predicting the output first.
- After each section, close the tab and explain the concept aloud in 2 minutes.
- Revisit Sections 7 (Closures), 14 (Event Loop), and 15–16 (Promises/async) at least 3 times — these appear in 80%+ of JS interviews.
- **Code examples:** Lines marked with `// comments` expand into **Line-by-line explanation** tables in the PDF — read both the code and the table.

Table of Contents

1. History & Role of JavaScript
2. Variables: var, let, const
3. Data Types, Coercion & Equality
4. Operators & Modern Syntax
5. Functions
6. Scope
7. Closures
8. The `this` Keyword
9. Prototypes & Classes
10. Arrays — Complete Reference
11. Objects — Complete Reference
12. Strings, Numbers, Math & Date
13. Error Handling
14. The Event Loop
15. Promises
16. async/await

- 17. [Fetch API & JSON](#)
- 18. [ES Modules vs CommonJS](#)
- 19. [Utility Patterns: Debounce, Throttle, Deep Clone, Curry](#)
- 20. [Interview Question Bank](#)
- 21. [Where to Practice](#)

1. History & Role of JavaScript

Theory

JavaScript was created by **Brendan Eich** at Netscape in **1995** in just **10 days**. Originally named "Mocha," then "LiveScript," it was renamed "JavaScript" for marketing reasons (Java was popular). Despite the name, JavaScript has almost nothing in common with Java.

Year	Milestone
1995	JavaScript created for Netscape Navigator
1997	ECMAScript (official JavaScript standard) standardization begins
2009	ECMAScript 5 – strict mode, JSON support
2015	ECMAScript 2015 (also called ES6) – classes, modules, arrow functions, promises
2015+	Annual ECMAScript releases (2016, 2017, ... 2024)
Today	Runs everywhere: browsers, servers (Node.js), mobile (React Native), desktop (Electron), Internet of Things

JavaScript's role in MERN stack:

- **MongoDB** — document database queried via JavaScript driver
- **Express** — JavaScript web framework on Node.js
- **React** — JavaScript user interface library (JSX syntax compiles to JavaScript)
- **Node.js** — JavaScript runtime outside the browser

Runtime Environments: Browser vs Node.js

Feature	Browser	Node.js
Global object	<code>window</code>	<code>global</code> (or <code>globalThis</code>)
Document Object Model access	Yes (<code>document</code> , <code>window</code>)	No
File system	No (sandboxed)	Yes (<code>fs</code> module)
Module system	ES Modules (+ bundlers)	CommonJS + ES Modules
Event loop	Web Application Programming Interfaces + microtasks	libuv thread pool + phases
<code>console</code>	DevTools console	Terminal stdout

```
// Browser-only
console.log(window.location.href);
document.getElementById('app');

// Node.js-only
const fs = require('fs'); // CommonJS
fs.readFileSync('./file.txt', 'utf8');

// Universal (ES2020+)
console.log(globalThis); // works in both environments
```

V8 Engine: Both Chrome and Node.js use Google's V8 engine to compile JavaScript to machine code. SpiderMonkey (Firefox) and JavaScriptCore (Safari) are other engines — behavior is standardized by ECMAScript, but subtle timing differences exist.

Common Mistakes

- Assuming JavaScript is "just a browser language" — Node.js powers Netflix, PayPal, LinkedIn backends.
- Confusing JavaScript with Java in interviews — instant red flag.
- Ignoring which runtime you're in (e.g., using `window` in Node without checking).

Interview Tips

- Be ready to explain: "JavaScript is single-threaded but non-blocking thanks to the event loop."

- Know that JS is **dynamically typed** and **multi-paradigm** (OOP, functional, procedural).
- Mention ECMAScript vs JavaScript: ECMAScript is the official specification; JavaScript is the language implementation in browsers and Node.js.

2. Variables: var, let, const

Theory

JavaScript provides three ways to declare variables:

Keyword	Scope	Hoisted	Reassignable	Redeclarable	Temporal Dead Zone
<code>var</code>	Function (or global)	Yes (initialized <code>undefined</code>)	Yes	Yes	No
<code>let</code>	Block	Yes (not initialized)	Yes	No	Yes
<code>const</code>	Block	Yes (not initialized)	No*	No	Yes

* `const` prevents reassignment of the binding, not mutation of object contents.

```
// var – function scoped (leaks out of if/for blocks)
function example() {
  if (true) {
    var x = 10;           // var is hoisted to function top; visible af
  }
  console.log(x);        // 10 – x "leaked" out of the if block
}

// let – block scoped (stays inside { })
function example2() {
  if (true) {
    let y = 10;           // y exists only inside this block
  }
  // console.log(y);      // ReferenceError – y is not defined outside
}

// const – must initialize; cannot reassign the variable name
const PI = 3.14159;      // Must assign immediately
// PI = 3;               // TypeError – cannot reassign const binding

const user = { name: 'Aarav' }; // const locks the reference, not the object
user.name = 'Vihaan';        // OK – mutating object property
// user = {};               // TypeError – cannot point user to a new obj
```

Line-by-line explanation (javascript)

Line	What it does
<code>var x = 10;</code>	var is hoisted to function top; visible after this block
<code>console.log(x);</code>	10 – x "leaked" out of the if block
<code>let y = 10;</code>	y exists only inside this block
<code>const PI = 3.14159;</code>	Must assign immediately
<code>const user = { name: 'Aarav' };</code>	const locks the reference, not the object
<code>user.name = 'Vihaan';</code>	OK – mutating object property

Hoisting

Hoisting is JavaScript's behavior of moving declarations to the top of their scope during the compilation phase.

```

console.log(a);           // undefined – var a was hoisted and set to undefined
var a = 5;                // Assignment happens here (after the log above)

// What the engine effectively does for var:
var a;                    // Declaration hoisted to top of scope
console.log(a);           // undefined (initialized as undefined)
a = 5;                    // Assignment stays in place

console.log(b);           // ReferenceError – in Temporal Dead Zone (cannot use before declaration line)
let b = 10;               // let is hoisted but NOT initialized until this line

```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(a);</code>	undefined – var a was hoisted and set to undefined
<code>var a = 5;</code>	Assignment happens here (after the log above)
<code>var a;</code>	Declaration hoisted to top of scope
<code>console.log(a);</code>	undefined (initialized as undefined)
<code>a = 5;</code>	Assignment stays in place
<code>console.log(b);</code>	ReferenceError – in Temporal Dead Zone (cannot use before declaration line)
<code>let b = 10;</code>	let is hoisted but NOT initialized until this line

Temporal Dead Zone: The period between entering scope and the `let` / `const` declaration line. Accessing the variable during the Temporal Dead Zone throws `ReferenceError`. (We do not use the short form "TDZ" in beginner sections.)

```

{
  // Temporal Dead Zone starts for `value` – name exists but cannot be accessed yet
  // console.log(value);           // ReferenceError if uncommented
  let value = 42;                 // Temporal Dead Zone ends here – value is in scope
  console.log(value);             // 42 – safe to use now
}

```

Line-by-line explanation (javascript)

Line	What it does
<code>let value = 42;</code>	Temporal Dead Zone ends here – value is initialized
<code>console.log(value);</code>	42 – safe to use now

Best Practices

1. Default to `const` — use `let` only when reassignment is needed.
2. Never use `var` in modern code (legacy codebases may still have it).
3. Declare variables at the top of their block for readability.
4. Use descriptive names: `isLoggedIn` , `totalPrice` , `fetchUserById` .

```
// Good
const MAX_RETRIES = 3;
let attemptCount = 0;

// Bad
var x = 3;
var y = 0;
```

Common Mistakes

```
// Mistake 1: var in loops
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // prints 3, 3, 3
}

// Fix with let
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 100); // prints 0, 1, 2
}

// Mistake 2: thinking const makes objects immutable
const arr = [1, 2];
arr.push(3); // works! arr is [1, 2, 3]
```

Interview Tips

- Explain hoisting with `var` vs `let` / `const` — interviewers love **Temporal Dead Zone** questions.
- Know that `typeof undeclaredVar` returns `"undefined"` but accessing it throws in strict mode for `let` / `const`.
- Be ready to draw scope chain diagrams for nested functions.

3. Data Types, Coercion & Equality

Theory – 8 Data Types

JavaScript has 7 primitive types + 1 reference type (Object):

Type	typeof result	Example
<code>undefined</code>	<code>"undefined"</code>	<code>let x;</code>
<code>boolean</code>	<code>"boolean"</code>	<code>true</code> , <code>false</code>
<code>number</code>	<code>"number"</code>	<code>42</code> , <code>3.14</code> , <code>NaN</code> , <code>Infinity</code>
<code>bigint</code>	<code>"bigint"</code>	<code>9007199254740991n</code>
<code>string</code>	<code>"string"</code>	<code>"hello"</code>
<code>symbol</code>	<code>"symbol"</code>	<code>Symbol('id')</code>
<code>null</code>	<code>"object"</code> ⚠️	<code>null</code>
<code>object</code>	<code>"object"</code>	<code>{}</code> , <code>[]</code> , <code>function</code> ⚠️

typeof Quirks

```
typeof null;           // "object" – famous bug, never fixed for compatibility
typeof [];             // "object" – arrays are objects in JS
typeof function(){};   // "function" – special case (functions are callable objects)
typeof NaN;            // "number" – NaN is still the number type
typeof undefined;      // "undefined"
typeof 42n;             // "bigint" – ES2020 big integers
typeof Symbol();       // "symbol" – unique identifier type
```

Line-by-line explanation (javascript)

Line	What it does
<code>typeof null;</code>	"object" – famous bug, never fixed for compatibility
<code>typeof [];</code>	"object" – arrays are objects in JS
<code>typeof function(){};</code>	"function" – special case (functions are callable objects)
<code>typeof NaN;</code>	"number" – NaN is still the number type
<code>typeof undefined;</code>	"undefined"
<code>typeof 42n;</code>	"bigint" – ES2020 big integers
<code>typeof Symbol();</code>	"symbol" – unique identifier type

Better null check:

```
value === null;           // strict
Object.prototype.toString.call(null); // "[object Null]"
```

Truthy and Falsy – Complete List

Falsy values (only 8):

Value	Notes
false	Boolean false
0	Number zero
-0	Negative zero
0n	BigInt zero
""	Empty string
null	Null
undefined	Undefined
NaN	Not a Number

Everything else is truthy, including:

```
Boolean([]);      // true - empty array is truthy!  
Boolean({});      // true - empty object is truthy!  
Boolean("0");      // true - string "0" is truthy  
Boolean("false");  // true - string "false" is truthy  
Boolean(function(){}); // true
```

Type Coercion

Implicit coercion happens when JS converts types automatically:

```
"5" + 3;           // "53" - + with string triggers string concat  
"5" - 3;           // 2 - - forces numeric conversion  
"5" * "2";         // 10 - * and / always convert to numbers  
true + 1;          // 2 - true coerces to 1 in numeric context  
false + 1;         // 1 - false coerces to 0  
[] + [];           // "" - both arrays become "" then concat  
[] + {};           // "[object Object]" - {} becomes string via  
{} + [];           // 0 or "[object Object]" - depends on parse
```

Line-by-line explanation (javascript)

Line	What it does
<code>"5" + 3;</code>	"53" – + with string triggers string concatenation
<code>"5" - 3;</code>	2 – - forces numeric conversion
<code>"5" * "2";</code>	10 – * and / always convert to numbers
<code>true + 1;</code>	2 – true coerces to 1 in numeric context
<code>false + 1;</code>	1 – false coerces to 0
<code>[] + [];</code>	"" – both arrays become "" then concat
<code>[] + {};</code>	"[object Object]" – {} becomes string via toString()
<code>{ } + [];</code>	0 or "[object Object]" – depends on parse context

Explicit coercion:

```
Number("42");    // 42
String(42);      // "42"
Boolean(1);       // true
parseInt("42px"); // 42
parseFloat("3.14"); // 3.14
```

== VS ===

Operator	Name	Behavior
<code>==</code>	Loose equality	Compares with type coercion
<code>===</code>	Strict equality	Compares type AND value, no coercion
<code>!=</code>	Loose inequality	Coercion applied
<code>!==</code>	Strict inequality	No coercion

```
// == coercion rules (memorize these for interviews)
null == undefined; // true
null == 0;         // false
undefined == 0;    // false
"5" == 5;          // true
"" == 0;           // true
false == 0;        // true
[] == false;       // true
[] == ![];         // true - infamous trick question

// === - always prefer this
"5" === 5;         // false
null === undefined; // false
NaN === NaN;       // false - use Number.isNaN()
```

`Object.is()` — SameValue comparison (handles `NaN` and `-0`):

```
Object.is(NaN, NaN); // true
Object.is(-0, +0);   // false
Object.is(5, 5);     // true
```

Common Mistakes

- Using `==` thinking it saves time — causes subtle bugs.
- Checking `if (arr)` to see if array is empty — `{}.length === 0` is correct.
- Comparing objects with `===` expecting deep equality — compares references only.

Interview Tips

- Recite all 8 falsy values from memory.
- Explain why `typeof null === "object"`.
- Know `Object.is` vs `===` difference for `NaN` and signed zero.

4. Operators & Modern Syntax

Arithmetic & Assignment Operators

```
let a = 10;
a += 5;    // 15
a **= 2;   // 225 (exponentiation, ES2016)
a ??= null; // assigns only if nullish (ES2021)

// Unary
++a;      // pre-increment
a++;      // post-increment
+a;       // unary plus (coerces to number)
!a;       // logical NOT
```

Logical Operators

```
// && – returns first falsy or last value
0 && "hello";    // 0
"hi" && "hello"; // "hello"

// || – returns first truthy or last value
0 || "default";  // "default"

// ?? – nullish coalescing (only null/undefined trigger default)
0 ?? "default";   // 0 (0 is not nullish!)
null ?? "default"; // "default"

// Combining
const name = user?.name ?? "Guest";
```

Optional Chaining (?.)

```
const user = {
  profile: {
    address: { city: "Bangalore" }
  }
};

user?.profile?.address?.city; // "Bangalore"
user?.profile?.phone?.number; // undefined (no error)

// With methods
obj.method?.(); // calls only if method exists

// With arrays
arr?.[0]; // safe index access
```

Template Literals

```
const name = "Priya";
const salary = 3000000;

// Basic
const msg = `Hello, ${name}!`;

// Multi-line
const html = `
  <div>
    <h1>${name}</h1>
    <p>CTC: ₹${(salary / 100000).toFixed(1)} LPA</p>
  </div>
`;

// Tagged templates
function highlight(strings, ...values) {
  return strings.reduce((acc, str, i) =>
    acc + str + (values[i] ? `<mark>${values[i]}</mark>` : ''), '');
}
highlight`Name: ${name}`; // "Name: <mark>Priya</mark>"
```

Nullish Coalescing vs Logical OR

Expression	<code> </code> result	<code>??</code> result
<code>0 10</code>	<code>10</code>	<code>0</code>
<code>"" "default"</code>	<code>"default"</code>	<code>""</code>
<code>null ?? 10</code>	<code>10</code>	<code>10</code>
<code>undefined ?? 10</code>	<code>10</code>	<code>10</code>
<code>false ?? true</code>	<code>false</code>	<code>false</code>

Common Mistakes

- Mixing `??` with `&&` / `||` without parentheses (SyntaxError in some cases).
- Using optional chaining when a null value should throw (fail-fast vs fail-safe).

Interview Tips

- Explain short-circuit evaluation in `&&` and `||`.
- Know that `??` only checks `null` and `undefined`, not other falsy values.

5. Functions

Function Declaration

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
// Hoisted – can call before declaration  
sayHi(); // works  
function sayHi() { console.log("Hi"); }
```

Function Expression

```
const greet = function(name) {  
  return `Hello, ${name}!`;  
};  
// Not hoisted  
// greet(); // works only after line above
```

Arrow Functions (ES6)

```
const add = (a, b) => a + b;  
const square = x => x * x;           // single param – parens optional  
const noop = () => { /* no return */ };  
const getObj = () => ({ key: 1 });    // return object – wrap in ()  
  
// Key differences from regular functions:  
// 1. No own `this` – inherits from enclosing scope  
// 2. No `arguments` object  
// 3. Cannot be used as constructors (no `new`)  
// 4. No `prototype` property
```

IIFE – Immediately Invoked Function Expression

```
(function() {  
  const secret = "hidden";  
  console.log("Runs immediately");  
})();  
  
// Modern alternative – block scope with let/const  
{  
  const secret = "hidden";  
}
```

Higher-Order Functions

A function that takes a function as argument or returns a function.

```
function operate(a, b, fn) {  
  return fn(a, b);  
}  
  
operate(5, 3, (x, y) => x + y); // 8  
  
// Built-in HOFs  
[1, 2, 3].map(x => x * 2);  
[1, 2, 3].filter(x => x > 1);  
[1, 2, 3].reduce((acc, x) => acc + x, 0);
```

Pure Functions

A pure function:

1. Always returns the same output for the same input.
2. Has no side effects (no mutation, no I/O, no console.log in strict definition).

```
// Pure  
const add = (a, b) => a + b;  
const double = arr => arr.map(x => x * 2);  
  
// Impure  
let count = 0;  
const increment = () => ++count; // side effect  
const addRandom = x => x + Math.random(); // non-deterministic  
const logAndReturn = x => { console.log(x); return x; };
```

Default Parameters & Rest Parameters

```
function createUser(name, role = "user", active = true) {  
  return { name, role, active };  
}  
  
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}  
  
sum(1, 2, 3, 4); // 10
```

Common Mistakes

- Using arrow functions as object methods when you need `this`.
- Using arrow functions as constructors.
- Forgetting that `return` on its own line in arrow functions with `{}` body needs explicit return.

Interview Tips

- Explain hoisting difference between declarations and expressions.
- Be ready to implement `map`, `filter`, `reduce` from scratch.
- Know when pure functions matter (React, Redux, testing).

6. Scope

Types of Scope

Scope Type	Created By	Accessible
Global	Outside all functions/blocks	Everywhere
Function	<code>function</code> keyword	Inside that function
Block	<code>{ }</code> with <code>let</code> / <code>const</code>	Inside that block
Module	ES Module file	Exported bindings only

```
const globalVar = "I'm global";

function outer() {
  const outerVar = "outer";

  function inner() {
    const innerVar = "inner";
    console.log(globalVar); // accessible
    console.log(outerVar); // accessible – lexical scope
    console.log(innerVar); // accessible
  }
  inner();
  // console.log(innerVar); // ReferenceError
}

outer();
```

Lexical Scope

JavaScript uses **lexical (static) scope** — a function's scope is determined by where it is *written*, not where it is *called*.

```
const x = "global";

function outer() {
  const x = "outer";
  function inner() {
    console.log(x); // "outer" – looks up lexical chain
  }
  return inner;
}

const fn = outer();
fn(); // "outer" – NOT "global"
```

Scope Chain

When a variable is accessed, JS looks in:

1. Current scope
2. Outer scope

3. ... up to global

4. If not found → `ReferenceError`

```
let a = 1;
function foo() {
  let b = 2;
  function bar() {
    let c = 3;
    console.log(a, b, c); // 1, 2, 3
  }
  bar();
}
```

Common Mistakes

- Assuming `if` blocks create function scope (only `var` leaks; `let` / `const` don't).
- Shadowing variables unintentionally:

```
let count = 0;
function update() {
  let count = 1; // shadows outer count
  // ...
}
```

Interview Tips

- Draw scope chain for nested functions — very common whiteboard task.
- Distinguish scope from context (`this` is context, not scope).

7. Closures

Definition

A **closure** is a function that retains access to variables from its **lexical scope** even after the outer function has finished executing.

```
function createCounter() {
  let count = 0; // enclosed variable

  return function() {
    count++;
    return count;
  };
}

const counter = createCounter();
counter(); // 1
counter(); // 2
counter(); // 3
// `count` is not accessible directly, but the inner function keeps it alive
```

Classic Examples

1. Private variables (module pattern):

```
function createBankAccount(initialBalance) {
  let balance = initialBalance;

  return {
    deposit(amount) { balance += amount; return balance; },
    withdraw(amount) {
      if (amount > balance) throw new Error("Insufficient funds");
      balance -= amount;
      return balance;
    },
    getBalance() { return balance; }
  };
}

const account = createBankAccount(1000);
account.deposit(500); // 1500
account.getBalance(); // 1500
// account.balance - undefined (private)
```

2. Function factories:

```
function multiply(factor) {  
  return (number) => number * factor;  
}  
  
const double = multiply(2);  
const triple = multiply(3);  
double(5); // 10  
triple(5); // 15
```

3. Event handlers / callbacks:

```
function setupButton(buttonId, message) {  
  document.getElementById(buttonId).addEventListener('click', () => {  
    alert(message); // closure over `message`  
  });  
}
```

4. Memoization:

```
function memoize(fn) {  
  const cache = new Map();  
  return function(...args) {  
    const key = JSON.stringify(args);  
    if (cache.has(key)) return cache.get(key);  
    const result = fn(...args);  
    cache.set(key, result);  
    return result;  
  };  
}  
  
const fib = memoize(function(n) {  
  if (n <= 1) return n;  
  return fib(n - 1) + fib(n - 2);  
});
```

Memory Leaks with Closures

Closures keep references alive. If you close over large objects unnecessarily, memory isn't freed.

```
// LEAK: hugeData stays in memory because closure references it
function leaky() {
  const hugeData = new Array(1e6).fill('x');
  return function() {
    console.log(hugeData[0]); // keeps entire array alive
  };
}

// FIX: only close over what you need
function fixed() {
  const hugeData = new Array(1e6).fill('x');
  const first = hugeData[0];
  return function() {
    console.log(first); // hugeData can be GC'd
  };
}
```

Common leak sources:

- Forgotten event listeners
- Timers (`setInterval`) referencing closures
- Closures stored in global variables

Interview Questions on Closures

```
// Q1: Classic loop + setTimeout
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 3, 3, 3

// Fix:
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 0, 1, 2

// Q2: Multiple closures sharing same variable
function createFunctions() {
  const result = [];
  for (var i = 0; i < 3; i++) {
    result.push(function() { return i; });
  }
  return result;
}
createFunctions().map(f => f()); // [3, 3, 3]

// Q3: Module pattern output
const modules = (function() {
  let privateVar = 0;
  return {
    increment: () => ++privateVar,
    get: () => privateVar
  };
})();
modules.increment();
modules.increment();
modules.get(); // 2
```

Interview Tips

- Define closure in one sentence: "A function bundled with its lexical environment."
- Always mention memory implications in senior interviews.
- Connect closures to React hooks (useState closure over state).

8. The `this` Keyword

Four Rules of `this` Binding

Rule	Condition	<code>this</code> value
1. Default	Plain function call	<code>globalThis</code> (strict: <code>undefined</code>)
2. Implicit	<code>obj.method()</code>	<code>obj</code>
3. Explicit	<code>call/apply/bind</code>	Specified object
4. <code>new</code>	<code>new Func()</code>	New empty object

```
// Rule 1: Default binding
function showThis() {
  console.log(this);
}
showThis(); // globalThis (browser: Window)

"use strict";
function strictThis() {
  console.log(this); // undefined
}

// Rule 2: Implicit binding
const user = {
  name: "Rahul",
  greet() {
    console.log(this.name);
  }
};
user.greet(); // "Rahul"

const greetFn = user.greet;
greetFn(); // undefined - lost implicit binding!

// Rule 3: Explicit binding
greetFn.call(user); // "Rahul"
greetFn.apply(user); // "Rahul"
const bound = greetFn.bind(user);
bound(); // "Rahul"

// Rule 4: new binding
function Person(name) {
  this.name = name;
}
const p = new Person("Ananya");
console.log(p.name); // "Ananya"
```

Arrow Functions and `this`

Arrow functions **do not have their own** `this` — they inherit from enclosing lexical scope.

```
const obj = {
  name: "Test",
  regular: function() { console.log(this.name); },
  arrow: () => { console.log(this.name); }
};
obj.regular(); // "Test"
obj.arrow();   // undefined (inherits outer `this`)
```

call, apply, bind – Implementations

```
// Polyfill for Function.prototype.call
Function.prototype.myCall = function(context, ...args) {
  context = context ?? globalThis;
  const sym = Symbol('fn');
  context[sym] = this;
  const result = context[sym](...args);
  delete context[sym];
  return result;
};

// Polyfill for Function.prototype.apply
Function.prototype.myApply = function(context, args = []) {
  return this.myCall(context, ...args);
};

// Polyfill for Function.prototype.bind
Function.prototype.myBind = function(context, ...boundArgs) {
  const fn = this;
  return function(...callArgs) {
    return fn.apply(context, [...boundArgs, ...callArgs]);
  };
};

// Usage
function introduce(city) {
  console.log(`${this.name} from ${city}`);
}
introduce.myCall({ name: "Kavya" }, "Mumbai");
const boundIntro = introduce.myBind({ name: "Kavya" }, "Delhi");
boundIntro(); // "Kavya from Delhi"
```

Common Mistakes

- Losing `this` when passing methods as callbacks — always `bind` or use arrow wrapper.

- Using arrow functions as object methods when `this` is needed.
- Confusing `this` with scope/closures.

Interview Tips

- Priority: `new` > explicit (`call/apply/bind`) > implicit > default.
- Implement `bind` from scratch — very common.
- Explain `this` in class methods vs arrow class fields in React components.

9. Prototypes & Classes

Prototype Chain

Every JavaScript object has an internal `[[Prototype]]` link (accessed via `__proto__` or `Object.getPrototypeOf()`).

```
const animal = {
  eats: true,
  walk() { console.log("walking"); }
};

const rabbit = Object.create(animal);
rabbit.hops = true;

rabbit.walk(); // "walking" – found on prototype
console.log(rabbit.eats); // true

// Chain: rabbit → animal → Object.prototype → null
```

```
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function() {
  return `Hi, I'm ${this.name}`;
};

const p = new Person("Dev");
p.greet(); // "Hi, I'm Dev"
console.log(p.__proto__ === Person.prototype); // true
console.log(Person.prototype.__proto__ === Object.prototype); // true
```

Object.create

```
const proto = {
  greet() { return `Hello, ${this.name}`; }
};

const user = Object.create(proto);
user.name = "Sam";
user.greet(); // "Hello, Sam"

// Object.create with property descriptors
const obj = Object.create(Object.prototype, {
  name: { value: "Fixed", writable: false, enumerable: true }
});
```

ES6 Classes

```
class Employee {
  // Private field (ES2022)
  #salary;

  // Static property
  static company = "TechCorp India";

  constructor(name, salary) {
    this.name = name;
    this.#salary = salary;
  }

  // Instance method
  getDetails() {
    return `${this.name} earns ${this.#salary}`;
  }

  // Getter / Setter
  get annualSalary() {
    return this.#salary * 12;
  }

  set annualSalary(val) {
    this.#salary = val / 12;
  }

  // Static method
  static createIntern(name) {
    return new Employee(name, 25000);
  }

  // Private method
  #calculateBonus() {
    return this.#salary * 0.1;
  }
}

class Manager extends Employee {
  constructor(name, salary, team) {
    super(name, salary);
    this.team = team;
  }

  getDetails() {
    return `${super.getDetails()}, manages ${this.team.length} people`;
  }
}
```

```
const mgr = new Manager("Neha", 200000, ["A", "B", "C"]);
mgr.getDetails();
```

Classes are syntactic sugar over prototypes:

```
typeof Employee; // "function"
Employee.prototype.constructor === Employee; // true
```

Common Mistakes

- Modifying `Object.prototype` directly — affects all objects globally.
- Confusing `__proto__` (instance link) with `prototype` (constructor property).
- Forgetting `super()` in derived class constructor.

Interview Tips

- Draw prototype chain diagram for `new` keyword step by step.
- Explain difference between `__proto__` and `prototype`.
- Implement inheritance without `class` using `Object.create`.

10. Arrays – Complete Reference

Creating Arrays

```
const arr1 = [1, 2, 3];
const arr2 = new Array(3); // [empty × 3] – sparse!
const arr3 = Array.from({ length: 3 }, (_, i) => i); // [0, 1, 2]
const arr4 = Array.of(7); // [7] – not [empty × 7]
const arr5 = [...arr1]; // shallow copy
```

Mutating Methods

Method	Returns	Description
<code>push(...items)</code>	new length	Add to end
<code>pop()</code>	removed item	Remove from end
<code>unshift(...items)</code>	new length	Add to start
<code>shift()</code>	removed item	Remove from start
<code>splice(start, deleteCount, ...items)</code>	deleted items	Add/remove at index
<code>sort(compareFn?)</code>	same array	Sort in place
<code>reverse()</code>	same array	Reverse in place
<code>fill(value, start?, end?)</code>	same array	Fill with value
<code>copyWithin(target, start, end?)</code>	same array	Copy section

```
const nums = [3, 1, 4, 1, 5];
nums.push(9);           // [3, 1, 4, 1, 5, 9]
nums.pop();             // [3, 1, 4, 1, 5]
nums.splice(1, 2, 99);  // [3, 99, 1, 5] – removed 1,4; inserted 99
nums.sort((a, b) => a - b); // numeric sort!
```

Non-Mutating Methods

Method	Returns	Use When
<code>concat(...arrays)</code>	new array	Merge arrays
<code>slice(start?, end?)</code>	new array	Extract portion
<code>indexOf(value)</code>	index or -1	Find first index
<code>lastIndexOf(value)</code>	index or -1	Find last index
<code>includes(value)</code>	boolean	Check existence
<code>find(predicate)</code>	element or undefined	Find first match
<code>findIndex(predicate)</code>	index or -1	Index of first match
<code>findLast(predicate)</code>	element or undefined	ES2023
<code>findLastIndex(predicate)</code>	index or -1	ES2023

Iteration Methods (Functional)

Method	Returns	Use When
<code>forEach(fn)</code>	undefined	Side effects only
<code>map(fn)</code>	new array	Transform each element
<code>filter(fn)</code>	new array	Keep matching elements
<code>reduce(fn, init?)</code>	accumulator	Aggregate to single value
<code>reduceRight(fn, init?)</code>	accumulator	Reduce right-to-left
<code>some(fn)</code>	boolean	Any element matches?
<code>every(fn)</code>	boolean	All elements match?
<code>flat(depth?)</code>	new array	Flatten nested arrays
<code>flatMap(fn)</code>	new array	map + flat(1)

```
const products = [
  { name: "Laptop", price: 65000, category: "electronics" },
  { name: "Book", price: 499, category: "education" },
  { name: "Phone", price: 45000, category: "electronics" },
];

// map – transform
const names = products.map(p => p.name);

// filter – select
const electronics = products.filter(p => p.category === "electronics");

// reduce – aggregate
const total = products.reduce((sum, p) => sum + p.price, 0);

// find – single item
const book = products.find(p => p.category === "education");

// some / every
products.some(p => p.price > 50000); // true
products.every(p => p.price > 0);    // true

// flatMap
const sentences = ["hello world", "foo bar"];
sentences.flatMap(s => s.split(" ")); // ["hello", "world", "foo", "bar"]
```

ES2023+ Array Methods

```
const arr = [1, 2, 3, 4, 5];

// toReversed, toSorted, toSpliced, with – non-mutating versions
arr.toReversed();           // [5, 4, 3, 2, 1]
arr.toSorted((a, b) => b - a); // [5, 4, 3, 2, 1]
arr.with(2, 99);            // [1, 2, 99, 4, 5] – replace index 2
```

Common Mistakes

- Using `sort()` without compare function on numbers: `[10, 2, 1].sort()` → `[1, 10, 2]`.
- Using `forEach` when you need to break early (use `for...of` or `some`).
- Confusing `map` (returns array) with `forEach` (returns undefined).

Interview Tips

- Implement `map`, `filter`, `reduce`, `flat` from scratch.
- Know time complexity: `push/pop` $O(1)$, `shift/unshift` $O(n)$, `sort` $O(n \log n)$.
- Explain sparse vs dense arrays.

11. Objects – Complete Reference

Creating Objects

```
// Object literal
const user = { name: "Arjun", age: 28 };

// Constructor
const obj = new Object();
obj.key = "value";

// Object.create
const proto = { greet() { return "hi"; } };
const child = Object.create(proto);

// Factory function
function createUser(name) {
  return { name, createdAt: new Date() };
}

// class (see Section 9)
```

Object Static Methods

```
const target = { a: 1 };
const source = { b: 2, c: 3 };

Object.assign(target, source); // { a: 1, b: 2, c: 3 } – shallow merge
Object.keys(obj);              // ["name", "age"]
Object.values(obj);            // ["Arjun", 28]
Object.entries(obj);           // [["name", "Arjun"], ["age", 28]]
Object.fromEntries([["a", 1]]); // { a: 1 }

"a" in obj;                    // true – checks prototype chain
obj.hasOwnProperty("name");    // true – own property only
Object.hasOwn(obj, "name");    // ES2022 – safer alternative
```

freeze, seal, preventExtensions

Method	Add	Delete	Modify	Reconfigure
preventExtensions	✗	✓	✓	✓
seal	✗	✗	✓	✗
freeze	✗	✗	✗	✗

```
const config = Object.freeze({ apiUrl: "https://api.example.com" });
config.apiUrl = "hacked"; // silently fails (strict mode: TypeError)
config.newKey = 1;        // fails

const sealed = Object.seal({ count: 0 });
sealed.count = 5;        // OK
delete sealed.count;     // fails
```

Property Descriptors

```
const obj = {};  
  
Object.defineProperty(obj, "id", {  
  value: 42,  
  writable: false,      // cannot change value  
  enumerable: false,    // hidden from for...in / Object.keys  
  configurable: false   // cannot delete or reconfigure  
});  
  
Object.defineProperties(obj, {  
  name: { value: "Test", writable: true, enumerable: true },  
  secret: { value: "hidden", enumerable: false }  
});  
  
const desc = Object.getOwnPropertyDescriptor(obj, "id");  
// { value: 42, writable: false, enumerable: false, configurable: false }
```

Destructuring

```
const user = { name: "Isha", age: 25, city: "Pune", role: "developer" };  
  
// Basic  
const { name, age } = user;  
  
// Rename  
const { name: userName } = user;  
  
// Default values  
const { country = "India" } = user;  
  
// Rest  
const { name: n, ...rest } = user; // rest = { age, city, role }  
  
// Nested  
const data = { user: { profile: { email: "a@b.com" } } };  
const { user: { profile: { email } } } = data;  
  
// Function parameters  
function greet({ name, age = 18 }) {  
  console.log(`${name} is ${age}`);  
}
```

Spread & Rest

```
// Spread – expand iterable
const defaults = { theme: "dark", lang: "en" };
const userPrefs = { lang: "hi", fontSize: 14 };
const settings = { ...defaults, ...userPrefs };
// { theme: "dark", lang: "hi", fontSize: 14 }

// Clone (shallow!)
const clone = { ...original };

// Rest in destructuring
const [first, ...remaining] = [1, 2, 3, 4]; // first=1, remaining=[2,3,4]

// Rest in function params
function log(type, ...messages) {
  messages.forEach(m => console.log(`[${type}] ${m}`));
}
```

Common Mistakes

- Thinking `{ ...obj }` is a deep clone — nested objects share references.
- Using `Object.freeze` expecting deep immutability — it's shallow only.
- Destructuring `undefined`: `const { x } = undefined` throws `TypeError`.

Interview Tips

- Implement shallow and deep clone functions.
- Explain difference between `in` operator and `hasOwnProperty`.
- Know property descriptor flags for defining reactive/getter properties.

12. Strings, Numbers, Math & Date

Strings

```
const str = "Hello, MERN Bootcamp!";

// Access
str[0];           // "H"
str.charAt(0);    // "H"
str.at(-1);       // "!" – ES2022, supports negative index

// Search
str.indexOf("MERN");    // 7
str.includes("Boot");   // true
str.startsWith("Hello"); // true
str.endsWith("!");      // true

// Extract
str.slice(7, 11);       // "MERN"
str.substring(7, 11);   // "MERN" – swaps if start > end
str.substr(7, 4);       // "MERN" – deprecated

// Transform
str.toLowerCase();
str.toUpperCase();
str.trim();             // remove whitespace both ends
str.trimStart();
str.replace("MERN", "Full Stack");
str.replaceAll("e", "E"); // ES2021

// Split & Join
str.split(", ");        // ["Hello", "MERN Bootcamp!"]
["a", "b"].join("-");   // "a-b"

// Template literals (see Section 4)
`Length: ${str.length}`; // 20

// Padding
"5".padStart(3, "0");    // "005"
"5".padEnd(3, "0");      // "500"

// Repeat
"ha".repeat(3);          // "hahaha"
```

Numbers

```
// Special values
Infinity;
-Infinity;
NaN; // result of invalid math: 0/0, "abc" * 2

Number.isNaN(NaN);      // true
Number.isNaN("hello");  // false (unlike global isNaN)
Number.isFinite(42);     // true
Number.isInteger(3.0);   // true

// Parsing
parseInt("42px", 10);    // 42 – always specify radix!
parseFloat("3.14em");    // 3.14
Number("42");            // 42
Number("");              // 0
Number(null);            // 0
Number(undefined);       // NaN

// Formatting
(1234567.89).toLocaleString("en-IN"); // "12,34,567.89"
(3000000).toLocaleString("en-IN", { style: "currency", currency: "INR" });
// "₹30,00,000.00"

// Safe integer range
Number.MAX_SAFE_INTEGER; // 9007199254740991 (2^53 - 1)
Number.MIN_SAFE_INTEGER;

// BigInt for larger integers
const big = 9007199254740991n + 1n;
```

Math

```
Math.PI;           // 3.141592653589793
Math.E;            // 2.718281828459045

Math.abs(-5);      // 5
Math.ceil(4.1);    // 5
Math.floor(4.9);   // 4
Math.round(4.5);   // 5
Math.trunc(4.9);   // 4

Math.max(1, 5, 3); // 5
Math.min(1, 5, 3); // 1
Math.pow(2, 10);   // 1024
2 ** 10;           // 1024 – exponentiation operator

Math.sqrt(16);     // 4
Math.cbrt(27);     // 3 – cube root
Math.hypot(3, 4);  // 5 –  $\sqrt{3^2 + 4^2}$ 

Math.random();     // 0 <= n < 1
// Random integer between min and max (inclusive)
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

Date

```
const now = new Date();
const specific = new Date("2026-07-30T10:00:00+05:30");
const fromParts = new Date(2026, 6, 30); // month is 0-indexed!

now.getFullYear(); // 2026
now.getMonth(); // 6 (July)
now.getDate(); // 30
now.getDay(); // 0-6 (Sun-Sat)
now.getHours();
now.getTime(); // Unix timestamp in ms

now.toISOString(); // "2026-07-30T..."
now.toLocaleDateString("en-IN"); // "30/7/2026"

// Timestamp
Date.now(); // current ms since epoch

// Difference in days
const diff = (date2 - date1) / (1000 * 60 * 60 * 24);
```

Common Mistakes

- Using `==` with NaN — always use `Number.isNaN()`.
- Forgetting months are 0-indexed in `Date` constructor.
- Using floating point for money: `0.1 + 0.2 !== 0.3` — use integers (paise) or libraries.

Interview Tips

- Explain IEEE 754 floating point limitations.
- Know string immutability — methods return new strings.
- For dates in production, mention libraries like `date-fns` or `Day.js`.

13. Error Handling

try / catch / finally

```
function parseJSON(str) {
  try {
    const data = JSON.parse(str);
    return data;
  } catch (error) {
    if (error instanceof SyntaxError) {
      console.error("Invalid JSON:", error.message);
    } else {
      throw error; // re-throw unexpected errors
    }
    return null;
  } finally {
    console.log("parseJSON completed"); // always runs
  }
}
```

finally runs regardless of try/catch outcome — useful for cleanup (closing connections, hiding loaders).

Throwing Errors

```
function withdraw(balance, amount) {
  if (amount <= 0) {
    throw new RangeError("Amount must be positive");
  }
  if (amount > balance) {
    throw new Error("Insufficient balance");
  }
  return balance - amount;
}
```

Custom Error Classes

```
class AppError extends Error {
  constructor(message, statusCode, code) {
    super(message);
    this.name = "AppError";
    this.statusCode = statusCode;
    this.code = code;
  }
}

class ValidationError extends AppError {
  constructor(message, field) {
    super(message, 400, "VALIDATION_ERROR");
    this.name = "ValidationError";
    this.field = field;
  }
}

class NotFoundError extends AppError {
  constructor(resource) {
    super(`${resource} not found`, 404, "NOT_FOUND");
    this.name = "NotFoundError";
  }
}

// Usage in Express-style handler
function getUser(id) {
  if (!id) throw new ValidationError("ID is required", "id");
  const user = db.find(id);
  if (!user) throw new NotFoundError("User");
  return user;
}

// Catching by type
try {
  getUser(null);
} catch (err) {
  if (err instanceof ValidationError) {
    console.log(err.field, err.message);
  } else if (err instanceof AppError) {
    console.log(err.statusCode, err.code);
  } else {
    throw err;
  }
}
```

Error Stack Traces

```
try {
  throw new Error("Something broke");
} catch (e) {
  console.log(e.message); // "Something broke"
  console.log(e.stack);   // full stack trace
  console.log(e.name);    // "Error"
}
```

Common Mistakes

- Empty catch blocks that swallow errors silently.
- Catching all errors without re-throwing unknown ones.
- Using strings instead of Error objects: `throw "bad"` — loses stack trace.

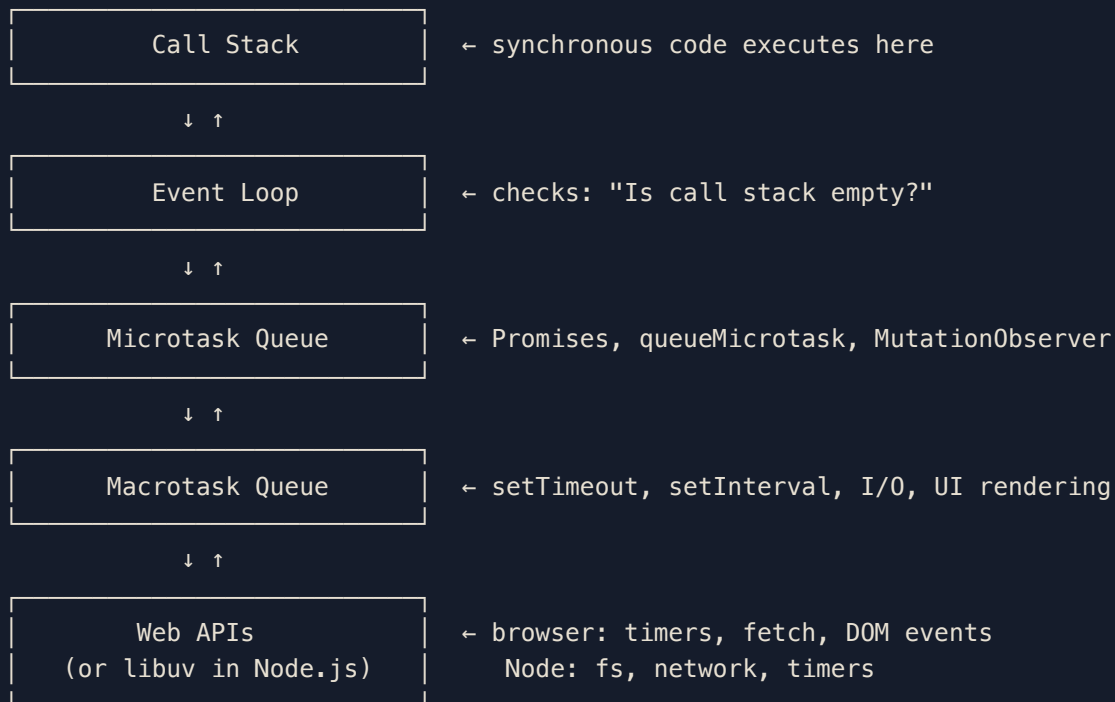
Interview Tips

- Explain difference between operational errors (expected) vs programmer errors (bugs).
- Know `Error.captureStackTrace` in Node.js for custom errors.
- Mention global handlers: `window.onerror`, `process.on('uncaughtException')`.

14. The Event Loop

Architecture Overview

JavaScript is **single-threaded** — one call stack, one thing at a time. Non-blocking I/O is achieved via the **event loop**.



Execution Order

1. Run synchronous code (call stack) until empty.
2. Run **all** microtasks (Promise callbacks, `queueMicrotask`).
3. Run **one** macrotask (e.g., one `setTimeout` callback).
4. Repeat from step 2.

Type	Examples	Priority
Synchronous	function calls, <code>console.log</code>	Immediate (call stack)
Microtask	<code>Promise.then</code> , <code>async/await</code> , <code>queueMicrotask</code>	After sync, before next macrotask
Macrotask	<code>setTimeout</code> , <code>setInterval</code> , <code>setImmediate</code> (Node), I/O	One per loop iteration

Node.js Event Loop Differences

Node.js has **phases** (timers → pending → idle → poll → check → close), plus:

- `process.nextTick` — runs **before** microtasks (even higher priority).
- `setImmediate` — runs in check phase, after I/O.

Priority in Node: Call Stack → process.nextTick → Microtasks → Macrotasks

Prediction Examples

Example 1 — Basics:

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2
```

Example 2 — Nested promises and timeouts:

```
console.log("start");
setTimeout(() => console.log("timeout"), 0);
Promise.resolve()
  .then(() => {
    console.log("promise1");
    Promise.resolve().then(() => console.log("promise2"));
  });
console.log("end");
// Output: start, end, promise1, promise2, timeout
```

Example 3 — async/await:

```
async function foo() {
  console.log("foo start");
  await Promise.resolve();
  console.log("foo end");
}
console.log("script start");
foo();
console.log("script end");
// Output: script start, foo start, script end, foo end
// (await splits: sync part runs immediately, rest goes to microtask)
```

Example 4 — Tricky mixed:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
setTimeout(() => console.log("C"), 0);
Promise.resolve().then(() => {
  console.log("D");
  setTimeout(() => console.log("E"), 0);
});
Promise.resolve().then(() => console.log("F"));
console.log("G");
// Output: A, G, D, F, B, C, E
```

Example 5 — async function chain:

```
async function async1() {
  console.log("1");
  await async2();
  console.log("2");
}
async function async2() {
  console.log("3");
}
console.log("4");
async1();
console.log("5");
// Output: 4, 1, 3, 5, 2
```

Example 6 — Node.js nextTick:

```
// Node.js only
console.log("start");
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("promise"));
process.nextTick(() => console.log("nextTick"));
console.log("end");
// Output: start, end, nextTick, promise, timeout
```

Example 7 — Interview classic:

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
Promise.resolve().then(() => console.log("promise"));  
// Output: promise, 3, 3, 3
```

Common Mistakes

- Thinking `setTimeout(fn, 0)` runs immediately — it waits for stack to clear + microtasks.
- Blocking the call stack with heavy sync code — freezes UI in browser.
- Not understanding that `await` yields to microtask queue.

Interview Tips

- Draw the event loop diagram from memory.
- Practice 10+ prediction questions — pattern recognition is key.
- Mention `requestAnimationFrame` (browser) runs before paint, separate from macrotasks.

15. Promises

Three States

State	Meaning	Transition
pending	Initial state	→ fulfilled or rejected
fulfilled	Operation succeeded	Final
rejected	Operation failed	Final

Once settled (fulfilled/rejected), a promise cannot change state.

Creating Promises

```
const promise = new Promise((resolve, reject) => {
  // Executor runs synchronously!
  const success = true;
  if (success) {
    resolve("Data loaded");
  } else {
    reject(new Error("Failed to load"));
  }
});

promise
  .then(value => console.log(value))
  .catch(error => console.error(error.message))
  .finally(() => console.log("Cleanup"));
```

Chaining

```
fetchUser(id)
  .then(user => fetchOrders(user.id))
  .then(orders => calculateTotal(orders))
  .then(total => console.log(`Total: ₹${total}`))
  .catch(err => console.error("Pipeline failed:", err));

// Each .then can return a value or a new Promise
// Return value → wrapped in Promise.resolve
// Thrown error → caught by next .catch
```

Promise Static Methods

```
// Promise.all – fails fast if ANY rejects
const [users, products] = await Promise.all([
  fetchUsers(),
  fetchProducts()
]);
// If one fails, entire Promise.all rejects

// Promise.allSettled – waits for ALL, never rejects
const results = await Promise.allSettled([
  fetchUsers(),
  fetchProducts()
]);
// [{ status: 'fulfilled', value: [...] }, { status: 'rejected', reason: Error }]

// Promise.race – first to settle wins
const fastest = await Promise.race([
  fetchFromServer1(),
  fetchFromServer2(),
  timeout(5000) // reject if > 5s
]);

// Promise.any – first to FULFILL wins (ES2021)
const firstSuccess = await Promise.any([
  fetchMirror1(),
  fetchMirror2()
]);
// Only rejects if ALL fail (AggregateError)
```

Method	Resolves When	Rejects When	Use Case
<code>all</code>	All fulfill	Any rejects	Parallel deps (all required)
<code>allSettled</code>	All settle	Never	Audit/report all outcomes
<code>race</code>	First settles	First rejects	Timeout patterns
<code>any</code>	First fulfills	All reject	Fallback mirrors

Promise Utilities

```
function timeout(ms) {
  return new Promise( (_, reject) =>
    setTimeout(() => reject(new Error(`Timeout after ${ms}ms`)), ms)
  );
}

function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

// Wrap callback API
function readFilePromise(path) {
  return new Promise((resolve, reject) => {
    fs.readFile(path, 'utf8', (err, data) => {
      if (err) reject(err);
      else resolve(data);
    });
  });
}
```

Common Mistakes

- Forgetting to `return` inside `.then` chain — breaks pipeline.
- Creating "promise hell" instead of chaining or using `async/await`.
- Not catching rejections — unhandled rejection warnings/crashes.
- Assuming `Promise.all` runs in parallel — it starts all immediately but JS is still single-threaded.

Interview Tips

- Implement `Promise.all`, `Promise.race` from scratch.
- Explain difference between microtask (Promise) and macrotask (`setTimeout`).
- Convert callback-based API to Promise (promisify pattern).

16. async/await

Basics

`async/await` is syntactic sugar over Promises.

```
async function fetchUserData(userId) {
  const response = await fetch(`/api/users/${userId}`);
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  const data = await response.json();
  return data;
}

// async function ALWAYS returns a Promise
fetchUserData(1).then(data => console.log(data));
```

Error Handling

```
// try/catch with async/await
async function loadData() {
  try {
    const data = await fetchData();
    return data;
  } catch (error) {
    console.error("Failed:", error.message);
    return null;
  }
}

// .catch on the returned promise
async function riskyOperation() {
  const result = await mightFail();
  return result;
}
riskyOperation().catch(err => console.error(err));
```

Sequential vs Parallel

```
// ❌ Sequential – slow (one after another)
async function sequential() {
  const user = await fetchUser();    // 200ms
  const orders = await fetchOrders(); // 200ms
  const products = await fetchProducts(); // 200ms
  // Total: ~600ms
}

// ✅ Parallel – fast (all at once)
async function parallel() {
  const [user, orders, products] = await Promise.all([
    fetchUser(),    // \
    fetchOrders(),  // all start together → ~200ms total
    fetchProducts() // /
  ]);
}

// Mixed – parallel where independent, sequential where dependent
async function mixed(userId) {
  const user = await fetchUser(userId);    // need user first
  const [orders, wishlist] = await Promise.all([
    fetchOrders(user.id),
    fetchWishlist(user.id)
  ]);
  return { user, orders, wishlist };
}
```

Top-Level await (ES Modules)

```
// module.js
const config = await fetchConfig();
export { config };
```

Common Mistakes

```
// Mistake: forEach with async – doesn't await!
async function processItems(items) {
  items.forEach(async (item) => {
    await processItem(item); // forEach doesn't wait!
  });
  console.log("Done"); // prints BEFORE items processed
}

// Fix: for...of or Promise.all
async function processItemsFixed(items) {
  for (const item of items) {
    await processItem(item); // sequential
  }
  // OR
  await Promise.all(items.map(item => processItem(item))); // parallel
}

// Mistake: unnecessary async
async function getValue() {
  return 42; // just return 42, no need for async
}
```

Interview Tips

- Explain that `await` pauses async function, not the main thread.
- Compare async/await readability vs Promise chains.
- Implement async retry logic with exponential backoff.

17. Fetch API & JSON

Fetch Basics

```
// GET request
async function getUsers() {
  const response = await fetch("https://api.example.com/users", {
    method: "GET",
    headers: {
      "Content-Type": "application/json",
      "Authorization": "Bearer token123"
    }
  });

  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }

  return response.json();
}

// POST request
async function createUser(userData) {
  const response = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(userData)
  });
  return response.json();
}

// PUT, PATCH, DELETE – same pattern, change method
```

Response Object

```
const response = await fetch(url);

response.status;      // 200, 404, 500
response.ok;          // true if 200-299
response.statusText;   // "OK"
response.headers;      // Headers object
response.url;          // final URL (after redirects)

const text = await response.text(); // raw text
const json = await response.json(); // parsed JSON
const blob = await response.blob(); // binary data
```

JSON Methods

```
const obj = { name: "Tanvi", skills: ["JS", "React", "Node"] };

// Serialize
const jsonString = JSON.stringify(obj);
// '{"name":"Tanvi","skills":["JS","React","Node"]}'

JSON.stringify(obj, null, 2); // pretty print

// Deserialize
const parsed = JSON.parse(jsonString);

// JSON limitations
JSON.stringify({ date: new Date() });
// date becomes ISO string, not Date object on parse

JSON.stringify({ fn: () => {} });
// functions, undefined, symbols are omitted
```

Fetch with AbortController (Timeout/Cancel)

```
async function fetchWithTimeout(url, ms = 5000) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), ms);

  try {
    const response = await fetch(url, { signal: controller.signal });
    clearTimeout(timeoutId);
    return response.json();
  } catch (error) {
    if (error.name === "AbortError") {
      throw new Error("Request timed out");
    }
    throw error;
  }
}
```

Common Mistakes

- Not checking `response.ok` — fetch doesn't reject on 404/500.
- Calling `response.json()` twice — body can only be consumed once.
- Not handling network errors (offline, CORS) separately from HTTP errors.

Interview Tips

- Explain CORS at high level (browser security, preflight OPTIONS).
- Compare fetch vs axios (interceptors, auto JSON, timeout).
- Know `JSON.parse` can throw — always wrap in try/catch.

18. ES Modules vs CommonJS

CommonJS (Node.js traditional)

```
// math.js
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }

module.exports = { add, subtract };
// OR
exports.add = add;

// app.js
const { add, subtract } = require('./math');
const math = require('./math'); // whole module object

// Dynamic, synchronous loading
const config = require(`./config/${env}.js`);
```

ES Modules (Modern standard)

```
// math.js
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }
export default class Calculator { /* ... */ }

// app.js
import Calculator, { add, subtract } from './math.js';
import * as math from './math.js';

// Re-export
export { add } from './math.js';

// Dynamic import (returns Promise)
const module = await import('./math.js');
```

Key Differences

Feature	CommonJS	ES Modules
Syntax	<code>require</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Loading	Synchronous	Asynchronous
Tree shaking	Limited	Yes (static analysis)
<code>this</code> at top level	<code>module.exports</code>	<code>undefined</code>
File extension	<code>.js</code> (implicit)	<code>.js</code> or <code>.mjs</code> (explicit in Node)
Browser support	Needs bundler	Native in modern browsers
<code>__dirname</code>	Available	Not available (use <code>import.meta.url</code>)

Node.js ESM Setup

```
// package.json
{
  "type": "module"
}
```

```
// __dirname equivalent in ESM
import { fileURLToPath } from 'url';
import { dirname } from 'path';
const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
```

Common Mistakes

- Mixing `require` and `import` in same file without understanding interop.
- Forgetting `.js` extension in Node ESM imports.
- Using named import for default export: `import { X }` vs `import X`.

Interview Tips

- Explain why ESM enables tree shaking (static structure).
- Know `"type": "module"` vs `.mjs` extension.
- Mention bundlers (Webpack, Vite, esbuild) bridge browser + Node.

19. Utility Patterns: Debounce, Throttle, Deep Clone, Curry

Debounce

Delays execution until after `wait` ms have passed since the **last** call. Use for: search input, resize handlers.

```
function debounce(fn, wait, immediate = false) {
  let timeoutId = null;

  return function debounced(...args) {
    const context = this;
    const callNow = immediate && !timeoutId;

    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => {
      timeoutId = null;
      if (!immediate) fn.apply(context, args);
    }, wait);

    if (callNow) fn.apply(context, args);
  };
}

// Usage
const searchAPI = debounce((query) => {
  console.log("Searching:", query);
}, 300);

// HTML: <input oninput="searchAPI(this.value)" />
// User types "react" → only one API call 300ms after last keystroke
```

Throttle

Ensures function runs at most once every `limit` ms. Use for: scroll events, mouse move, button spam prevention.

```
function throttle(fn, limit) {
  let inThrottle = false;
  let lastArgs = null;
  let lastContext = null;

  return function throttled(...args) {
    const context = this;

    if (!inThrottle) {
      fn.apply(context, args);
      inThrottle = true;

      setTimeout(() => {
        inThrottle = false;
        if (lastArgs) {
          fn.apply(lastContext, lastArgs);
          lastArgs = null;
          lastContext = null;
        }
      }, limit);
    } else {
      lastArgs = args;
      lastContext = context;
    }
  };
}

// Usage
const handleScroll = throttle(() => {
  console.log("Scroll position:", window.scrollY);
}, 200);
window.addEventListener("scroll", handleScroll);
```

Deep Clone

```
function deepClone(value, seen = new WeakMap()) {
  // Primitives and functions
  if (value === null || typeof value !== "object") {
    return value;
  }

  // Handle circular references
  if (seen.has(value)) {
    return seen.get(value);
  }

  // Date
  if (value instanceof Date) {
    return new Date(value.getTime());
  }

  // RegExp
  if (value instanceof RegExp) {
    return new RegExp(value.source, value.flags);
  }

  // Map
  if (value instanceof Map) {
    const cloned = new Map();
    seen.set(value, cloned);
    value.forEach((v, k) => cloned.set(deepClone(k, seen), deepClone(v, seen)));
    return cloned;
  }

  // Set
  if (value instanceof Set) {
    const cloned = new Set();
    seen.set(value, cloned);
    value.forEach(v => cloned.add(deepClone(v, seen)));
    return cloned;
  }

  // Array or Object
  const cloned = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, cloned);

  for (const key of Reflect.ownKeys(value)) {
    cloned[key] = deepClone(value[key], seen);
  }

  return cloned;
}
```

```
// Usage
const original = { a: 1, b: { c: 2 }, d: [3, { e: 4 }] };
original.self = original; // circular
const copy = deepClone(original);
copy.b.c = 99;
console.log(original.b.c); // 2 – unchanged

// Modern alternative (no functions, dates, circular refs):
// structuredClone(original) – built-in, ES2022
```

Curry

Transforms a function with N arguments into N functions each taking one argument (or partial application).

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return (...nextArgs) => curried(...args, ...nextArgs);
  };
}

// Usage
function add(a, b, c) {
  return a + b + c;
}

const curriedAdd = curry(add);
curriedAdd(1)(2)(3);    // 6
curriedAdd(1, 2)(3);    // 6
curriedAdd(1)(2, 3);    // 6

// Practical example – reusable validators
const makeValidator = curry((rule, field, value) => rule(value) ? null : `${field}`);
const minLength = (n) => (str) => str.length >= n;
const validatePassword = makeValidator(minLength(8));
validatePassword("password", "123"); // "password invalid"
validatePassword("password", "secure123"); // null
```

Debounce vs Throttle Summary

Pattern	Behavior	Analogy	Use Case
Debounce	Wait for pause	Elevator door waits for last person	Search autocomplete
Throttle	Max once per interval	ATM allows one txn per 5 sec	Scroll position tracker

Common Mistakes

- Using debounce for scroll (misses events) — use throttle.
- Using throttle for search input (fires too often) — use debounce.
- Deep clone with `JSON.parse(JSON.stringify())` — loses Date, Function, undefined, handles no circular refs.

Interview Tips

- Implement all four from scratch in 20 minutes total — common live coding task.
- Mention `structuredClone` and lodash alternatives in production.
- Explain curry's role in functional programming and partial application.

20. Interview Question Bank

Q1: What is the output?

```
console.log(typeof typeof 1);
```

Answer: `"string"`. `typeof 1` returns `"number"`, and `typeof "number"` returns `"string"`.

Q2: Explain closure and fix this code

```
for (var i = 0; i < 5; i++) {  
  setTimeout(() => console.log(i), i * 1000);  
}
```

Answer: Prints `5` five times at intervals. `var` is function-scoped, all closures share same `i`. Fix: use `let i` (block-scoped) or IIFE `(function(j) { setTimeout(..., j) })(i)`.

Q3: What is the output?

```
const obj = {  
  name: "JS",  
  greet: function() { console.log(this.name); },  
  arrowGreet: () => { console.log(this.name); }  
};  
obj.greet();  
obj.arrowGreet();  
const fn = obj.greet;  
fn();
```

Answer: `"JS"`, `undefined`, `undefined`. Regular function gets `obj` as `this`; arrow inherits lexical `this` (global/module); detached method loses binding.

Q4: Implement Array.prototype.flat

```
function flat(arr, depth = 1) {  
  if (depth <= 0) return arr.slice();  
  return arr.reduce((acc, item) => {  
    return acc.concat(Array.isArray(item) ? flat(item, depth - 1) : item);  
  }, []);  
}
```

Q5: Event loop – predict output

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

Answer: 1, 4, 3, 2 . Sync first, then microtasks, then macrotasks.

Q6: Difference between null and undefined?

Answer:

	undefined	null
Meaning	Variable declared but not assigned	Intentional absence of value
typeof	"undefined"	"object" (bug)
JSON	Omitted	Preserved as null
Default	Function params without args, missing object keys	Explicitly set by developer

Q7: What is prototypal inheritance?

Answer: Objects inherit directly from other objects via the prototype chain (`[[Prototype]]`). When a property isn't found on the object, JS walks up the chain until `null` . ES6 classes are syntactic sugar over this mechanism. `Object.create(proto)` sets prototype explicitly.

Q8: Implement Promise.all

```
function promiseAll(promises) {  
  return new Promise((resolve, reject) => {  
    if (!Array.isArray(promises)) {  
      return reject(new TypeError("Argument must be array"));  
    }  
    const results = [];  
    let completed = 0;  
    if (promises.length === 0) return resolve([]);  
  
    promises.forEach((promise, index) => {  
      Promise.resolve(promise)  
        .then(value => {  
          results[index] = value;  
          completed++;  
          if (completed === promises.length) resolve(results);  
        })  
        .catch(reject);  
    });  
  });  
}
```

Q9: What is the difference between `==` and `===`?

Answer: `==` performs type coercion before comparison (`"5" == 5` is true). `===` checks strict equality without coercion (`"5" === 5` is false). Always prefer `===` except for null checks (`value == null` catches both null and undefined).

Q10: Explain event delegation

Answer: Attach one event listener to a parent element instead of many on children. Events bubble up; use `event.target` to identify which child was clicked. Benefits: fewer listeners, works for dynamically added elements, better memory usage.

```
document.getElementById("list").addEventListener("click", (e) => {  
  if (e.target.matches("li")) {  
    console.log("Clicked:", e.target.textContent);  
  }  
});
```

Q11: What are JavaScript memory leaks and common causes?

Answer: Memory that's no longer needed but not garbage collected.

1. **Global variables** — accidental globals without `var` / `let` / `const` .
2. **Forgotten timers** — `setInterval` never cleared.
3. **Closures** — holding references to large objects.
4. **Detached DOM nodes** — removed from DOM but referenced in JS.
5. **Event listeners** — not removed when component unmounts.

Q12: Implement bind from scratch

```
Function.prototype.myBind = function(context, ...boundArgs) {  
  const fn = this;  
  return function(...callArgs) {  
    // Support `new` — if called with new, ignore context override  
    if (new.target) return new fn(...boundArgs, ...callArgs);  
    return fn.apply(context, [...boundArgs, ...callArgs]);  
  };  
};
```

Q13: Difference between microtask and macrotask?

Answer: Microtasks (Promises, queueMicrotask) run after current script ends, before next render/macrotask. Macrotasks (setTimeout, setInterval, I/O) run one per event loop tick. Microtasks can starve macrotasks if chained infinitely.

Q14: What is hoisting?

Answer: During compilation phase, JS moves variable and function declarations to top of scope. `var` declarations are hoisted and initialized as `undefined`. `let` / `const` are hoisted but not initialized (TDZ). Function declarations are fully hoisted (can call before line). Function expressions follow variable hoisting rules.

Q15: How would you flatten a deeply nested array `[1, [2, [3, [4]]]]` ?

Answer:

```
// Recursive
function flatten(arr) {
  return arr.reduce((acc, val) =>
    acc.concat(Array.isArray(val) ? flatten(val) : val), []);
}

// Iterative with stack
function flattenIterative(arr) {
  const stack = [...arr];
  const result = [];
  while (stack.length) {
    const next = stack.pop();
    if (Array.isArray(next)) stack.push(...next);
    else result.push(next);
  }
  return result.reverse();
}

// ES2019+
[1, [2, [3, [4]]]].flat(Infinity); // [1, 2, 3, 4]
```

21. Where to Practice

Official Documentation

Resource	URL	Best For
MDN Web Docs	https://developer.mozilla.org/en-US/docs/Web/JavaScript	Authoritative reference, every API
ECMAScript Spec	https://tc39.es/ecma262/	Deep spec reading (advanced)

Structured Learning

Resource	URL	Best For
JavaScript.info	https://javascript.info/	Best free structured tutorial, modern ES6+
freeCodeCamp JS	https://www.freecodecamp.org/learn/javascript-algorithms-and-data-structures/	Free certification, hands-on exercises
Eloquent JavaScript	https://eloquentjavascript.net/	Deep conceptual book with exercises

DSA & Interview Practice

Resource	URL	Best For
LeetCode (JS)	https://leetcode.com/problemset/javascript/	Interview DSA in JavaScript
HackerRank JS	https://www.hackerrank.com/domains/tutorials/10-days-of-javascript	10-day JS challenge
Codewars	https://www.codewars.com/?language=javascript	Kata-style JS problems
GreatFrontEnd	https://www.greatfrontend.com/questions/javascript	Frontend-specific JS questions

Event Loop Visualizers

Resource	URL
Loupe (Philip Roberts)	http://latentflip.com/loupe/
JS Event Loop Visualizer	https://www.jsv9000.app/

Weekly Practice Plan (Post-Bootcamp)

Day	Activity	Time
Mon	javascript.info – 1 chapter + notes	1.5 hr
Tue	3 LeetCode Easy/Medium (JS)	1.5 hr
Wed	Re-implement 1 utility (debounce, curry, etc.)	1 hr
Thu	5 event loop prediction problems	45 min
Fri	2 interview questions aloud (Section 20)	1 hr
Sat	Build mini project (vanilla JS)	3 hr
Sun	Review weak areas from week	1 hr

Quick Reference Cheat Sheet

TRUTHY/FALSY: falsy = false, 0, -0, 0n, "", null, undefined, NaN

SCOPE: var → function | let/const → block

THIS: default < implicit < explicit < new

EVENT LOOP: sync → microtasks (all) → macrotask (one) → repeat

PROMISES: pending → fulfilled | rejected (immutable once settled)

ASYNC: async fn returns Promise; await pauses fn, not thread

CLONE: shallow = spread/assign | deep = structuredClone/custom

EQUALITY: prefer === | Object.is for NaN | == null for null+undefined

Next Module: 02-DOM & Browser APIs — Document manipulation, events, storage, and performance.

Remember: JavaScript mastery for 30 LPA roles isn't about memorizing syntax — it's about understanding *execution*, *memory*, and *async behavior*. Code every example. Predict before you run. Explain before you move on.

Last updated: July 2026 | MERN Mastery Bootcamp — Module 01

Product Company Interview Bank – JavaScript (100 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart, Salesforce, Stripe **Rule:** Full terms only — no unexplained abbreviations. Write for absolute beginners who know nothing.

Question 1: What is hoisting in JavaScript, and how does it behave for `var`, `let`, and `const` ?

Answer: Hoisting is JavaScript's behavior of moving declarations to the top of their scope during compilation, before code runs. Variables declared with `var` are hoisted and initialized as `undefined`, so you can reference them before the assignment line (though the value is `undefined`). Variables declared with `let` and `const` are also hoisted, but they stay in the Temporal Dead Zone until their declaration line runs — accessing them before that throws a `ReferenceError`. At companies like Google and Amazon, interviewers often follow up by asking you to predict output of a snippet mixing `var` and function declarations.

Question 2: What is the Temporal Dead Zone, and why does it exist?

Answer: The Temporal Dead Zone is the period between the start of a scope and the line where a `let` or `const` variable is declared. During this time, the variable name exists in the scope but cannot be accessed — doing so throws a `ReferenceError`. It exists to catch bugs from using variables before they are initialized, which `var` silently allowed. Interviewers at Meta and Stripe often ask candidates to explain why `typeof undeclaredVar` returns `"undefined"` but `typeof letVarBeforeDeclaration` throws an error.

Question 3: What will this code print, and why?

```
console.log(a);  
var a = 10;  
console.log(a);
```

Answer: It prints `undefined`, then `10`. The `var a` declaration is hoisted to the top of the scope and initialized to `undefined`, so the first `console.log` sees `undefined`. After `a = 10` runs, the second log prints `10`. This pattern is a classic hoisting trap asked at Adobe and Flipkart to test whether you understand declaration versus assignment.

Question 4: What is a closure in JavaScript?

Answer: A closure is when a function remembers and can access variables from the scope where it was created, even after that outer scope has finished running. The inner function "closes over" those variables, keeping them alive in memory. Closures power patterns like private state, callbacks, and factory functions. At Uber and Salesforce, follow-up questions often ask you to use a closure to implement a counter or memoization cache.

Question 5: What will this code output?

```
for (var i = 0; i < 3; i++) {  
  setTimeout(function () {  
    console.log(i);  
  }, 100);  
}
```

Answer: It prints `3` three times. Because `var` is function-scoped (not block-scoped), there is only one shared `i`. By the time the timeouts run, the loop has finished and `i` is `3`. This is one of the most common JavaScript interview questions at Amazon and Microsoft; the fix is to use `let` (which creates a new binding per iteration) or an immediately invoked function expression to capture each value.

Question 6: Fix the loop so it prints 0, 1, 2 using `var` .

Answer: Wrap the timeout callback in an immediately invoked function expression that captures the current index:

```
for (var i = 0; i < 3; i++) {  
  (function (j) {  
    setTimeout(function () {  
      console.log(j); // prints 0, then 1, then 2  
    }, 100);  
  })(i);  
}
```

Each immediately invoked function expression gets its own copy of `j` at the time the loop iteration runs. Interviewers at Google may also accept `let` as the modern answer and ask you to compare both approaches.

Question 7: What does the `this` keyword refer to in JavaScript?

Answer: `this` refers to the object that is the current execution context of a function — essentially, "who called this function." For a regular function called as `obj.method()`, `this` is `obj`. For a standalone function call, `this` is `undefined` in strict mode or the global object (`window` in browsers) in non-strict mode. Arrow functions do not have their own `this`; they inherit `this` from the enclosing scope. Product companies like Atlassian frequently test `this` binding with `call`, `apply`, and `bind`.

Question 8: What will this code print?

```
const obj = {
  name: "Stripe",
  greet: function () {
    console.log(this.name);
  },
  greetArrow: () => {
    console.log(this.name);
  }
};
obj.greet();
obj.greetArrow();
```

Answer: `greet()` prints `"Stripe"` because `this` inside a regular method refers to `obj`. `greetArrow()` prints `undefined` (or the global name if one exists) because arrow functions inherit `this` from the outer lexical scope — here, the module or global scope, not `obj`. This distinction is heavily tested at Meta and Stripe during frontend interviews.

Question 9: Explain the difference between `call`, `apply`, and `bind`.

Answer: All three methods let you explicitly set the `this` value of a function. `call(thisArg, arg1, arg2, ...)` invokes the function immediately with individual arguments. `apply(thisArg, [arg1, arg2, ...])` invokes immediately but accepts arguments as an array. `bind(thisArg, arg1, ...)` returns a new function with `this` permanently bound; it does not invoke immediately. At Amazon and Flipkart, a common follow-up is: "Implement your own version of `bind`."

Question 10: What is prototypal inheritance in JavaScript?

Answer: In JavaScript, objects can inherit properties and methods from other objects through a hidden link called the prototype. Every object has an internal `[[Prototype]]` (accessible via `Object.getPrototypeOf` or the deprecated `__proto__`). When you access a property, JavaScript looks on the object first, then walks up the prototype chain until it finds the property or reaches `null`. Classes in modern JavaScript are syntactic sugar over this prototype system. Google and Microsoft often ask candidates to explain how inheritance works without using the `class` keyword.

Question 11: What is the difference between `__proto__` and `prototype`?

Answer: `prototype` is a property on constructor functions (like `function Person() {}`) — it is the object that will become the `[[Prototype]]` of instances created with `new Person()`. `__proto__` (deprecated; use `Object.getPrototypeOf` instead) is the actual prototype link on any object instance pointing to its parent. In short: `prototype` is on functions; the instance's internal prototype points to that object. This distinction appears in debugging rounds at Adobe and Atlassian.

Question 12: What will this code output?

```
function Animal(name) {  
  this.name = name;  
}  
Animal.prototype.speak = function () {  
  return this.name + " makes a sound";  
};  
const dog = new Animal("Rex");  
console.log(dog.speak());  
console.log(dog.hasOwnProperty("speak"));  
console.log("speak" in dog);
```

Answer: It prints `"Rex makes a sound"`, then `false`, then `true`. `speak` lives on `Animal.prototype`, not directly on `dog`, so `hasOwnProperty("speak")` is `false`. The `in` operator checks the entire prototype chain, so `"speak" in dog` is `true`. This tests prototype chain lookup, a staple at Flipkart and Uber backend-to-fullstack interviews.

Question 13: Explain the JavaScript event loop in simple terms.

Answer: JavaScript is single-threaded — it runs one piece of code at a time. The event loop is the mechanism that lets JavaScript handle asynchronous work without blocking. When async tasks finish (timers, network responses, user events), their callbacks go into task queues. The event loop repeatedly checks: "Is the call stack empty? If yes, pull the next task from the queue and run it." Microtasks (like Promise callbacks) run before macrotasks (like `setTimeout`). Every major product company — Google, Meta, Amazon — asks event loop questions.

Question 14: In what order will this code log?

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

Answer: Order: **1** , **4** , **3** , **2** . Synchronous code runs first (**1** , **4**). Then the microtask queue drains — the Promise `.then` callback logs **3** . Finally the macrotask queue runs — `setTimeout` logs **2** , even with zero delay. Interviewers at Stripe and Salesforce use this exact pattern to test microtask versus macrotask priority.

Question 15: What is the difference between a microtask and a macrotask?

Answer: Microtasks are short, high-priority async callbacks scheduled by Promises, `queueMicrotask` , `MutationObserver` , and `async/await` continuations. Macrotasks (also called tasks) come from `setTimeout` , `setInterval` , input events, and network callbacks. After each macrotask completes, the engine runs all pending microtasks before picking the next macrotask. Understanding this ordering is critical for debugging race conditions in production code at companies like Uber and Atlassian.

Question 16: What is a Promise in JavaScript?

Answer: A Promise is an object representing the eventual result of an asynchronous operation — either success (fulfilled) or failure (rejected). It has three states: pending, fulfilled, and rejected. Once settled, a Promise cannot change state. You attach handlers with `.then()` for success, `.catch()` for errors, and `.finally()` for cleanup. Promises replaced callback pyramids and are foundational to modern async code at every company on this list.

Question 17: Implement a function `delay(ms)` that returns a Promise resolving after `ms` milliseconds.

Answer:

```
function delay(ms) {  
  return new Promise(function (resolve) {  
    setTimeout(resolve, ms);  
  });  
}  
// Usage: await delay(1000); // waits 1 second
```

This wraps `setTimeout` in a Promise so it works with `async/await`. At Amazon and Microsoft, interviewers often extend this to: "Implement `Promise.all`" or "Retry a fetch three times with delay between attempts."

Question 18: What does `async/await` do, and how is it related to Promises?

Answer: `async/await` is syntactic sugar built on Promises that makes asynchronous code look and read like synchronous code. An `async` function always returns a Promise. The `await` keyword pauses execution inside an `async` function until the Promise settles, then returns its value. Under the hood, `await` is equivalent to `.then()`. Error handling uses `try/catch` instead of `.catch()`. This pattern is standard in React and Node.js codebases at Meta, Google, and Adobe.

Question 19: What will this `async` function return?

```
async function fetchData() {  
  return 42;  
}  
const result = fetchData();  
console.log(result);
```

Answer: It logs `Promise { 42 }` (or similar Promise object), not `42`. An `async` function always wraps its return value in a resolved Promise. To get `42`, you must use `await fetchData()` inside another `async` function or call `fetchData().then(console.log)`. This catches many candidates at Flipkart and Stripe who assume `async` changes the return type to the raw value.

Question 20: What is type coercion in JavaScript?

Answer: Type coercion is JavaScript's automatic conversion of a value from one type to another during operations. It happens with the abstract equality operator (`==`), string concatenation

with `+`, and boolean contexts (`if` conditions). For example, `"5" + 1` becomes `"51"` (string concatenation), but `"5" - 1` becomes `4` (numeric subtraction forces conversion). Explicit conversion uses functions like `Number()`, `String()`, and `Boolean()`. Interviewers at Google and Amazon love asking candidates to predict coercion output.

Question 21: What are the results of these comparisons?

```
console.log([] == false);  
console.log([] == ![]);  
console.log(null == undefined);  
console.log(null === undefined);
```

Answer: `[] == false` is `true` — the empty array coerces to `""`, then to `0`; `false` coerces to `0`. `[] == ![]` is `true` — `![]` is `false` (arrays are truthy, so negation gives `false`), then same coercion chain leads to `0 == 0`. `null == undefined` is `true` (special rule in the abstract equality spec). `null === undefined` is `false` (strict equality checks type and value; they are different types). These "trick" questions appear in Adobe and Atlassian screening rounds.

Question 22: Why should you prefer `===` over `==` in production code?

Answer: The strict equality operator (`===`) compares both value and type without coercion, making behavior predictable and reducing hidden bugs. The abstract equality operator (`==`) performs type coercion, which can produce surprising results (like `0 == false` being `true`). Using `===` everywhere except rare null-checks (`value == null` catches both `null` and `undefined`) is a best practice enforced by linters at Stripe, Salesforce, and most product companies.

Question 23: What are arrow functions, and when should you NOT use them?

Answer: Arrow functions (introduced in ECMAScript 2015) provide shorter syntax and lexically bind `this`, `arguments`, `super`, and `new.target` from the enclosing scope. Do not use arrow functions as object methods when you need dynamic `this` (use regular functions). Do not use them as constructors (`new arrowFn()` throws). Do not use them where you need the `arguments` object. At Meta and Uber, interviewers ask when arrow functions cause bugs in React class components versus functional components.

Question 24: Explain destructuring assignment with an example.

Answer: Destructuring lets you unpack values from arrays or properties from objects into distinct variables in one statement.

```
const user = { name: "Priya", role: "engineer", level: 5 };
const { name, role } = user; // name = "Priya", role = "engineer"

const scores = [90, 85, 78];
const [first, , third] = scores; // first = 90, third = 78 (skip second)
```

It also supports defaults, renaming, and nested destructuring. This ECMAScript 2015 feature is used daily in React props and API response handling at every target company.

Question 25: What is the spread operator, and how does it differ from rest parameters?

Answer: The spread operator (`...`) expands an iterable into individual elements. Rest parameters use the same syntax but collect remaining elements into an array. Spread: `const merged = [...arr1, ...arr2]` copies and combines arrays. Rest: `function sum(first, ...rest) { return rest.reduce((a, b) => a + b, first); }` gathers extra arguments. Both were added in ECMAScript 2015. Amazon and Microsoft frequently ask candidates to clone objects or merge configurations using spread.

Question 26: What are template literals, and why are they better than string concatenation?

Answer: Template literals use backticks (```) and allow embedded expressions with `${expression}`, multiline strings without escape characters, and tagged templates for custom processing. Example: ``Hello, ${user.name}! You have ${count} messages.``. They are more readable and less error-prone than `"Hello, " + user.name + "!"`. Every modern JavaScript codebase at Google, Adobe, and Flipkart uses template literals as the default for string building.

Question 27: What is the difference between `map`, `filter`, and `reduce`?

Answer: All three are array methods that take a callback and do not mutate the original array.

`map` transforms each element and returns a new array of the same length. `filter` returns a new array containing only elements that pass a test. `reduce` accumulates all elements into a single value (sum, object, max, etc.). Example: `[1,2,3].map(x => x * 2) → [2,4,6]`; `.filter(x => x > 1) → [2,3]`; `.reduce((sum, x) => sum + x, 0) → 6`. These are asked in almost every JavaScript interview round.

Question 28: Implement a function that flattens a nested array one level deep without using `.flat()`.

Answer:

```
function flattenOneLevel(arr) {  
  return arr.reduce(function (acc, item) {  
    return acc.concat(Array.isArray(item) ? item : [item]);  
  }, []);  
}  
// flattenOneLevel([1, [2, 3], 4]) → [1, 2, 3, 4]
```

This uses `reduce` and `concat` to merge sub-arrays. At Google and Uber, follow-ups include: "Flatten to any depth recursively" or "What is the linear time complexity of your solution?" — meaning the runtime grows proportionally with the number of elements.

Question 29: What is `Set` and when would you use it over an array?

Answer: A `Set` is a collection of unique values — adding a duplicate has no effect. Lookups, additions, and deletions are fast (average constant time). Use a `Set` when you need uniqueness (deduplication, tracking visited nodes in a graph) or fast membership checks. Use an array when order matters for repeated elements or you need index-based access. At Stripe and Salesforce, a common question is: "Remove duplicates from an array" — `[...new Set(arr)]` is the idiomatic one-liner.

Question 30: What is a `Map` and how is it different from a plain object?

Answer: A `Map` stores key-value pairs where keys can be any type (objects, functions, primitives), not just strings or symbols. It maintains insertion order, has a `.size` property, and performs better for frequent additions and deletions. Plain objects are better for fixed-shape records with string keys and JSON serialization. Atlassian and Amazon interviewers ask when you would choose `Map` over `{}` — common answers include caching with object keys or counting frequencies.

Question 31: What is debouncing, and implement a debounce function.

Answer: Debouncing ensures a function runs only after a pause in calls — useful for search inputs, resize handlers, and auto-save. If the user types rapidly, the function waits until they stop for a specified delay.

```
function debounce(fn, delay) {  
  let timerId;  
  return function (...args) {  
    clearTimeout(timerId);  
    timerId = setTimeout(function () {  
      fn.apply(this, args);  
    }, delay);  
  };  
}
```

Google and Adobe frontend interviews almost always include debounce or throttle implementation questions.

Question 32: What is throttling, and how does it differ from debouncing?

Answer: Throttling guarantees a function runs at most once per time interval, regardless of how many times it is triggered. Debouncing waits for a pause; throttling enforces a regular cadence. Use throttle for scroll events, mouse movement tracking, or rate-limiting API calls. Use debounce for search-as-you-type or form validation after the user stops typing. At Uber and Flipkart, interviewers ask you to choose between the two for a given user interface scenario and justify your choice.

Question 33: Implement a throttle function.

Answer:

```
function throttle(fn, interval) {  
  let lastCall = 0;  
  return function (...args) {  
    const now = Date.now();  
    if (now - lastCall >= interval) {  
      lastCall = now;  
      fn.apply(this, args);  
    }  
  };  
}
```

This version fires immediately on the first call, then ignores calls until the interval passes. A follow-up at Meta might ask for a trailing-edge throttle that also fires once after the last event in a burst.

Question 34: What is memoization, and implement a basic memoize function?

Answer: Memoization caches the results of expensive function calls so repeated calls with the same arguments return the cached value instantly.

```
function memoize(fn) {  
  const cache = new Map();  
  return function (...args) {  
    const key = JSON.stringify(args);  
    if (cache.has(key)) return cache.get(key);  
    const result = fn.apply(this, args);  
    cache.set(key, result);  
    return result;  
  };  
}
```

This pattern is used in React (`useMemo`), dynamic programming, and data-fetching layers. Google and Amazon ask candidates to discuss cache invalidation and memory trade-offs as follow-ups.

Question 35: How does garbage collection work in JavaScript?

Answer: JavaScript engines automatically reclaim memory that is no longer reachable. The most common algorithm is mark-and-sweep: starting from roots (global object, call stack references), the engine marks all reachable objects, then sweeps (frees) everything unmarked. Objects become unreachable when no variable or closure references them. Memory leaks happen when you accidentally keep references — common causes include forgotten event listeners, global variables, and closures holding large objects. Microsoft and Salesforce system design rounds touch on memory when discussing long-lived single-page applications.

Question 36: What causes a memory leak in JavaScript, and give an example?

Answer: A memory leak occurs when memory that is no longer needed stays allocated because something still references it. Example: attaching an event listener inside a component but never removing it when the component is destroyed — the listener closure keeps the entire component state alive. Another example: storing data in a global array that grows forever (like an unbounded log cache). At Atlassian and Adobe, interviewers ask how you would detect leaks using browser DevTools Memory tab and heap snapshots.

Question 37: What is the difference between shallow copy and deep copy?

Answer: A shallow copy creates a new object or array but copies nested references — changes to nested objects affect both copies. A deep copy recursively clones everything, producing fully independent data. Shallow: `const copy = { ...original }` or `const copy = Object.assign({}, original)`. Deep: `structuredClone(original)` (modern browsers and Node.js) or `JSON.parse(JSON.stringify(original))` (limited — no functions, dates, or `undefined`). Amazon and Flipkart ask this when discussing immutable state patterns in React or Redux.

Question 38: What will this code output?

```
const original = { a: 1, nested: { b: 2 } };
const shallow = { ...original };
shallow.nested.b = 99;
console.log(original.nested.b);
```

Answer: It prints `99`. The spread operator creates a shallow copy — `shallow` is a new top-level object, but `shallow.nested` points to the same inner object as `original.nested`. Mutating the nested property affects both. This is a frequent debugging question at Stripe and Meta when discussing React state updates — always copy nested objects when immutability is required.

Question 39: What is strict mode (`"use strict"`), and what does it change?

Answer: Strict mode is an opt-in variant of JavaScript that catches common mistakes and disables unsafe features. It is activated with `"use strict"` at the top of a file or function. Changes include: assigning to undeclared variables throws `ReferenceError`; duplicate parameter names are syntax errors; `this` is `undefined` in standalone functions (not the global object); deleting non-configurable properties throws. All ECMAScript modules are strict by default. Google and Microsoft expect candidates to write interview code in strict mode.

Question 40: What are JavaScript modules (`import` / `export`), and how do they differ from script tags?

Answer: ECMAScript modules (`import` / `export`) let you split code into reusable files with explicit dependencies. Modules are automatically in strict mode, have their own scope (no global pollution), and support static analysis for tree-shaking (removing unused code at build time). Classic script tags share the global scope and load in order. Dynamic `import()` returns a Promise for lazy loading. Every modern frontend at Meta, Google, and Adobe uses module bundlers (Webpack, Vite) built on this system.

Question 41: What is event delegation, and why is it useful?

Answer: Event delegation attaches a single event listener to a parent element instead of one listener per child. When an event bubbles up, the handler checks `event.target` to determine which child was clicked. It is useful for dynamic lists (items added or removed after page load), better memory usage, and simpler code. Example: one `click` listener on a `<ul>` handles clicks on all `<li>` items. Flipkart and Uber product pages with long scrollable lists use this pattern extensively.

Question 42: What is the difference between event bubbling and event capturing?

Answer: When an event fires on a nested element, it travels in three phases: capturing (from root down to target), target (the element itself), and bubbling (from target back up to root). By default, listeners run during the bubbling phase. You can listen during capturing by passing `{ capture: true }` as the third argument to `addEventListener`. Calling `event.stopPropagation()` prevents the event from traveling further. Adobe and Atlassian ask this when debugging click handlers on nested user interface elements.

Question 43: Implement a function `once(fn)` that ensures `fn` runs only one time.

Answer:

```
function once(fn) {
  let called = false;
  let result;
  return function (...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);
    }
    return result;
  };
}
```

The closure tracks whether the function has been called. Subsequent calls return the cached result without re-invoking. This pattern appears in initialization logic, lazy singletons, and React's `useRef`-based run-once effects. Google interviewers may ask you to implement a version that also handles arguments differently on subsequent calls.

Question 44: What is currying in JavaScript?

Answer: Currying transforms a function that takes multiple arguments into a series of functions each taking one argument (or a few). Example: `const add = (a) => (b) => (c) => a + b + c`; `add(1)(2)(3)` returns `6`. It enables partial application — fixing some arguments to create

specialized functions. Functional programming libraries and some React hooks patterns use currying. At Stripe and Salesforce, follow-ups ask you to write a generic `curry` utility function.

Question 45: What will this Promise chain output?

```
Promise.resolve(1)
  .then((val) => val + 1)
  .then((val) => { throw new Error("fail"); })
  .then((val) => console.log("success:", val))
  .catch((err) => console.log("caught:", err.message))
  .then(() => console.log("done"));
```

Answer: It prints `"caught: fail"`, then `"done"`. The third `.then` throws, so the next `.then` (success handler) is skipped and execution jumps to `.catch`. After catching, the chain continues — the final `.then` runs because `.catch` returns a resolved Promise. Understanding Promise chain recovery is essential for error handling in production APIs at Amazon and Microsoft.

Question 46: Implement `Promise.all` from scratch.

Answer:

```
function promiseAll(promises) {
  return new Promise(function (resolve, reject) {
    const results = [];
    let completed = 0;
    if (promises.length === 0) return resolve([]);
    promises.forEach(function (promise, index) {
      Promise.resolve(promise)
        .then(function (value) {
          results[index] = value;
          completed++;
          if (completed === promises.length) resolve(results);
        })
        .catch(reject); // one failure rejects entire batch
    });
  });
}
```

This preserves result order regardless of completion order. Google and Meta frequently ask for this plus a discussion of `Promise.allSettled` (wait for all, never reject early) and `Promise.race`.

Question 47: What is the difference between `null` and `undefined` ?

Answer: `undefined` means a variable has been declared but not assigned a value, or a function returns nothing (implicit return). It is the default for missing object properties and function parameters. `null` is an intentional assignment representing "no value" or "empty." `typeof undefined` is `"undefined"`; `typeof null` is `"object"` (a long-standing language bug). Use `null` when you explicitly want to signal emptiness; `undefined` usually appears automatically. This basics question still appears at every company as a warm-up.

Question 48: What are Symbols in JavaScript, and why were they added?

Answer: A Symbol is a primitive type introduced in ECMAScript 2015 that creates guaranteed-unique identifiers. Every `Symbol("desc")` is distinct, even with the same description. Symbols are used for hidden object properties (not shown in `for...in` or `Object.keys`), well-known constants like `Symbol.iterator`, and preventing naming collisions in libraries. At Google and Atlassian, this comes up when discussing iterable protocols and meta-programming in frameworks.

Question 49: Debug this code – why does the counter always show 0?

```
function Counter() {  
  this.count = 0;  
}  
Counter.prototype.increment = () => {  
  this.count++;  
};  
const c = new Counter();  
c.increment();  
console.log(c.count);
```

Answer: The bug is using an arrow function for `increment`. Arrow functions do not have their own `this` — `this.count++` modifies the outer scope's `this` (likely the global object or `undefined` in strict mode), not the `Counter` instance. Fix: use a regular function — `Counter.prototype.increment = function () { this.count++; };`. This is a classic debugging question at Meta, Uber, and Flipkart that combines prototypes with `this` binding.

Question 50: You are building a search feature for a Flipkart-style e-commerce site. The user types in a search box and you call an API on each keystroke.

How do you optimize this, and what JavaScript concepts apply?

Answer: Use debouncing to wait until the user pauses typing (for example, 300 milliseconds) before firing the API call – this reduces hundreds of requests to a few. Cancel or ignore stale responses when a newer search term is already in flight (use an `AbortController` or a request identifier check). Cache recent search results in a `Map` keyed by query string for instant repeat searches. Show a loading state while the Promise is pending. These patterns combine debouncing, Promises, `async/await`, and memory-conscious caching – exactly the kind of practical system design question asked at Flipkart, Amazon, Google, and Uber in frontend interviews.

Questions 51-100 (top product companies)

Question 51: What is the difference between shallow copy and deep copy?

Answer: A shallow copy (`Object.assign`, spread) copies the top level; nested objects stay shared. A deep copy duplicates the graph (`structuredClone`). Product bugs: mutating a copied cart item also mutates the original. Google and Flipkart ask you to pick the right copy for a state update.

Question 52: Explain `map`, `filter`, and `reduce` and when reduce is the wrong tool.

Answer: `map` transforms 1:1, `filter` keeps some, `reduce` folds to any shape. Prefer `map` / `filter` when that is all you need — `reduce` that builds an array is harder to read. Meta FE screens often ask you to rewrite a `reduce` as `map`.

Question 53: What is optional chaining and nullish coalescing?

Answer: `a?.b` short-circuits if `a` is nullish. `??` uses the right side only for `null` / `undefined` (not `0` or `""`). Amazon likes `??` vs `||` as a trap.

Question 54: How do ES modules differ from CommonJS?

Answer: ES modules are static, live bindings, `import` / `export`, async in browsers. CommonJS is `require`, copy of exports, sync. Mixing them needs default-import interop. Node interviews at Uber ask about circular imports.

Question 55: What is the event loop `requestAnimationFrame` vs `setTimeout` for animation?

Answer: `requestAnimationFrame` runs before the next paint (~60 Hz) and pauses in background tabs. `setTimeout(16)` drifts and wastes work off-screen. Google FE expects this for jank talk.

Question 56: How would you implement `Promise.all` yourself?

Answer: Return a Promise; count completions; on any reject, reject immediately; on all fulfill, resolve the values in order. Meta and Google ask this after `Promise` basics.

Question 57: What is a memory leak from a forgotten timer?

Answer: A `setInterval` closure holds DOM nodes or large arrays. Clear on unmount / route change. Chrome heap snapshots at Google FE.

Question 58: Explain `async` / `await` error handling vs `.catch`.

Answer: `try/catch` around `await`. Forgotten `await` swallows errors into an unhandled rejection. Always return/await the Promise in Express. Adobe and Amazon ask this with a broken snippet.

Question 59: What are tagged template literals used for?

Answer: A function receives the string parts and interpolations — `sql ...` or ``css`...`. Used in GraphQL and CSS-in-JS. Atlassian may ask how they prevent injection if misused.

Question 60: How does `Object.freeze` differ from `const` ?

Answer: `const` blocks reassignment of the binding, not mutation of the object. `freeze` makes own properties non-writable (shallow). Stripe and Microsoft use this for config objects.

Question 61: What is the difference between `for...of` and `for...in` ?

Answer: `for...in` enumerates enumerable keys (including inherited). `for...of` uses the iterator (values). Never `for...in` arrays in production. Flipkart warm-up.

Question 62: Explain tail-call optimization – does JavaScript have it?

Answer: Most engines do not reliably TCO. Recursion can stack-overflow; prefer iteration or trampolines. Google may mention it to see if you claim ES6 TCO as fact.

Question 63: How do you share data between browser tabs?

Answer: `localStorage` + `storage` event, `BroadcastChannel`, or a `SharedWorker`. Cookies work but are sent to the server. Uber FE follow-up on logout-all-tabs.

Question 64: What is CORS and when does the browser send a preflight?

Answer: Cross-Origin Resource Sharing. Preflight (`OPTIONS`) for non-simple methods/headers. Server must allow origin, methods, headers. Never `Access-Control-Allow-Origin: *` with credentials. Amazon and Flipkart.

Question 65: How would you polyfill `bind` ?

Answer: Return a function that applies the original with a fixed `this` and prepended args, using `apply`. Classic Amazon / Flipkart coding-on-whiteboard.

Question 66: What is the difference between `slice` and `splice` ?

Answer: `slice` copies a range, no mutate. `splice` mutates and can insert/delete. Accidental `splice` in React state is a Meta bug question.

Question 67: Explain the module pattern vs ES modules for privacy.

Answer: IIFE + closures hid privates before ES modules. Modules scope is file-private; export only the public API. Still asked at Adobe.

Question 68: What is `queueMicrotask` vs `Promise.then` ?

Answer: Same microtask queue. `queueMicrotask` is the primitive; `then` also creates a Promise. Order among microtasks is enqueue order. Google snippet questions.

Question 69: How do you detect a circular reference when stringifying?

Answer: Keep a `WeakSet` of seen objects; skip or replace cycles. `JSON.stringify` throws on cycles. Amazon debugging.

Question 70: What is the difference between `undefined` holes in arrays and `empty` ?

Answer: `arr[1] = undefined` vs a hole from `delete` / sparse arrays. `map` skips holes. Microsoft likes this edge.

Question 71: Explain `new` – what does it do step by step?

Answer: Create object, set prototype to `Ctor.prototype`, call Ctor with `this`, return object unless Ctor returns an object. Google prototype round.

Question 72: What are WeakMap use cases?

Answer: Private fields keyed by object, DOM node metadata that should not prevent garbage collection. Atlassian / Google.

Question 73: How does `setTimeout(fn, 0)` actually delay?

Answer: It waits for the current stack + all microtasks + the timer phase. Not zero milliseconds. Event-loop follow-up at Meta.

Question 74: What is a thenable?

Answer: Any object with a `then` method. `Promise.resolve` will assimilate it. Used in old libraries; can be a security footgun. Rare Google.

Question 75: How do you implement debounce with leading and trailing options?

Answer: Leading fires immediately then locks; trailing fires after quiet. Lodash API. Flipkart search box.

Question 76: Explain `structuredClone` vs `JSON` clone for `Date` and `Map`.

Answer: `structuredClone` keeps Date and Map. JSON turns Date into a string and drops Map. Adobe / Uber.

Question 77: What is the difference between `element.innerHTML` and `textContent` ?

Answer: `innerHTML` parses HTML (XSS risk). `textContent` is text only. Always `textContent` for user strings. All FE security.

Question 78: How do service workers change fetch behavior?

Answer: They intercept fetch, can cache-first or network-first. Used for offline and performance. Google FE / PWA talk — high level is enough.

Question 79: What is the difference between `call` stack overflow and a memory leak?

Answer: Stack overflow is too-deep sync recursion. A leak is heap growth over time. Diagnose differently (stack vs heap snapshot). Google.

Question 80: Explain `Intl` for currency and dates in an e-commerce app.

Answer: `Intl.NumberFormat` / `DateTimeFormat` with locale. Do not hand-roll ₹ formatting. Flipkart / Amazon India.

Question 81: How would you sandbox user-provided HTML?

Answer: Do not use `innerHTML`. Use a sanitizer (DOMPurify) or render as text. CSP as defense in depth. Atlassian / Google.

Question 82: What is the iterator protocol for a range object?

Answer: Implement `[Symbol.iterator]()` returning `{ next }`. Enables `for...of` and spread. Google / Atlassian.

Question 83: Why is `typeof NaN === "number"` and how do you test NaN?

Answer: `Number.isNaN` (not global `isNaN` which coerces). Microsoft warm-up.

Question 84: How do you abort multiple fetches when a component unmounts?

Answer: One `AbortController`; pass `signal` to all; `abort()` in cleanup. React + Node interviews.

Question 85: What is the difference between `localStorage` and `sessionStorage`?

Answer: `localStorage` persists; `sessionStorage` dies with the tab. Both sync and ~5 MB. Do not store tokens. All FE.

Question 86: Explain event capturing vs bubbling.

Answer: Capture root→target, then bubble target→root. `addEventListener(type, fn, true)` for capture. Delegation uses bubble. Meta / Flipkart.

Question 87: How do you measure a function's time complexity from a snippet?

Answer: Loops, nested loops, log factors from halving, extra space from arrays/maps. They want Big-O spoken, not a lecture. All DSA+JS.

Question 88: What is `document.fragment` used for?

Answer: Build a subtree off-DOM, then one insert — fewer reflows. Older vanilla question; still at Adobe.

Question 89: How does `async` change the return type of a function?

Answer: It always returns a Promise. `return 1` → Promise of 1. Throwing rejects. Amazon snippet.

Question 90: What is the difference between `export default` and named exports?

Answer: One default per module; names are explicit and refactor-friendly. Prefer named in large apps. Atlassian.

Question 91: How would you implement a tiny pub/sub in the browser?

Answer: Map of event → Set of fns; `on` / `off` / `emit`. Used for cross-widget events. Uber FE.

Question 92: What is `performance.now` vs `Date.now` ?

Answer: `performance.now` is monotonic high-res; `Date.now` can jump. Use performance for benchmarks. Google FE.

Question 93: Explain same-origin policy in one minute.

Answer: Scheme + host + port must match to read another page's DOM/storage. CORS and `postMessage` are the escapes. All security.

Question 94: How do you prevent click-jacking?

Answer: CSP `frame-ancestors`, `X-Frame-Options`. Amazon / Adobe security.

Question 95: What is a generator used for in Redux-saga or similar?

Answer: Pause on effects (`yield take`), testable async flows. Mention only if they use sagas. Some Adobe.

Question 96: How do you clone a function's behavior with a different `this` ?

Answer: `bind`, or wrap and `apply`. Do not mutate the original. Amazon.

Question 97: What breaks if you use `parseInt` without a radix?

Answer: Old engines guess octal for leading 0. Always `parseInt(s, 10)`. Microsoft.

Question 98: How would you implement `once(fn)` ?

Answer: A closure flag; first call runs `fn`, later calls return the first result. Closure classic at Salesforce.

Question 99: What is the difference between `undefined` and not passing an argument?

Answer: Missing args are `undefined`. `fn.length` still counts declared params. Default params apply. Adobe.

Question 100: Search-as-you-type: list every JavaScript concept you would mention.

Answer: Debounce, AbortController, Map cache, Promise race/ignore stale, accessibility keyboard. Flipkart / Uber / Atlassian — same as Q50 but they expect this list in 30 seconds.